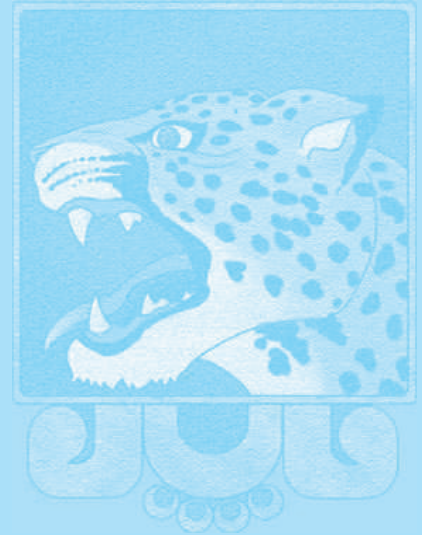


# MULTITHREADING AND PARALLEL PROGRAMMING

## Objectives

- To get an overview of multithreading (§32.2).
- To develop task classes by implementing the **Runnable** interface (§32.3).
- To create threads to run tasks using the **Thread** class (§32.3).
- To control threads using the methods in the **Thread** class (§32.4).
- To control animations using threads (§32.5, §32.7).
- To run code in the event dispatch thread (§32.6).
- To execute tasks in a thread pool (§32.8).
- To use synchronized methods or blocks to synchronize threads to avoid race conditions (§32.9).
- To synchronize threads using locks (§32.10).
- To facilitate thread communications using conditions on locks (§§32.11–32.12).
- To use blocking queues to synchronize access to an array queue, linked queue, and priority queue (§32.13).
- To restrict the number of accesses to a shared resource using semaphores (§32.14).
- To use the resource-ordering technique to avoid deadlocks (§32.15).
- To describe the life cycle of a thread (§32.16).
- To create synchronized collections using the static methods in the **Collections** class (§32.17).
- To develop parallel programs using the Fork/Join Framework (§32.18).



## 32.1 Introduction

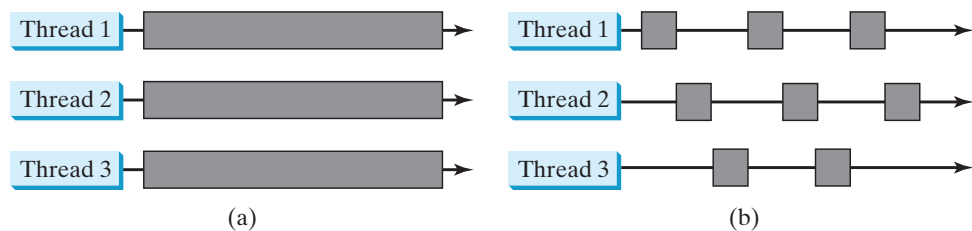
*Multithreading enables multiple tasks in a program to be executed concurrently.*

One of the powerful features of Java is its built-in support for *multithreading*—the concurrent running of multiple tasks within a program. In many programming languages, you have to invoke system-dependent procedures and functions to implement multithreading. This chapter introduces the concepts of threads and how to develop multithreading programs in Java.

## 32.2 Thread Concepts

*A program may consist of many tasks that can run concurrently. A thread is the flow of execution, from beginning to end, of a task.*

A *thread* provides the mechanism for running a task. With Java, you can launch multiple threads from a program concurrently. These threads can be executed simultaneously in multi-processor systems, as shown in Figure 32.1a.



**FIGURE 32.1** (a) Here multiple threads are running on multiple CPUs. (b) Here multiple threads share a single CPU.

In single-processor systems, as shown in Figure 32.1b, the multiple threads share CPU time, known as *time sharing*, and the operating system is responsible for scheduling and allocating resources to them. This arrangement is practical, because most of the time the CPU is idle. It does nothing, for example, while waiting for the user to enter data.

Multithreading can make your program more responsive and interactive, as well as enhance performance. For example, a good word processor lets you print or save a file while you are typing. In some cases, multithreaded programs run faster than single-threaded programs even on single-processor systems. Java provides exceptionally good support for creating and running threads and for locking resources to prevent conflicts.

When your program executes as an application, the Java interpreter starts a thread for the `main` method. When your program executes as an applet, the Web browser starts a thread to run the applet. You can create additional threads to run concurrent tasks in the program. In Java, each task is an instance of the `Runnable` interface, also called a *runnable object*. A *thread* is essentially an object that facilitates the execution of a task.

**32.1** Why is multithreading needed? How can multiple threads run simultaneously in a single-processor system?

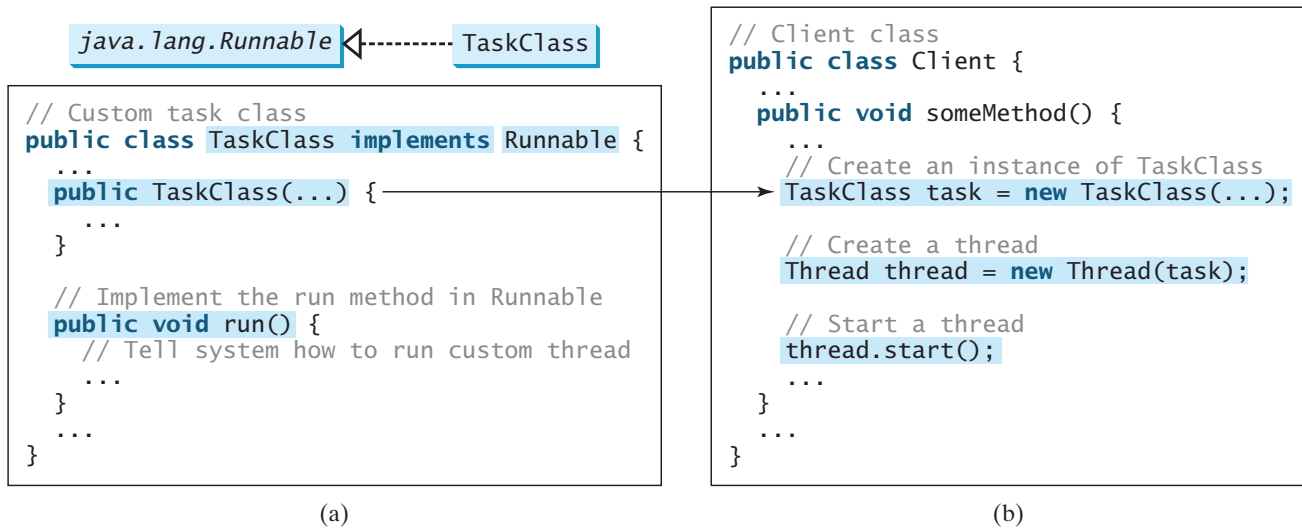
**32.2** What is a runnable object? What is a thread?

## 32.3 Creating Tasks and Threads

*A task class must implement the `Runnable` interface. A task must be run from a thread.*

Tasks are objects. To create tasks, you have to first define a class for tasks, which implements the `Runnable` interface. The `Runnable` interface is rather simple. All it contains is

the **run** method. You need to implement this method to tell the system how your thread is going to run. A template for developing a task class is shown in Figure 32.2a.



**FIGURE 32.2** Define a task class by implementing the **Runnable** interface.

Once you have defined a **TaskClass**, you can create a task using its constructor. For example,

```
TaskClass task = new TaskClass(...);
```

A task must be executed in a thread. The **Thread** class contains the constructors for creating threads and many useful methods for controlling threads. To create a thread for a task, use

```
Thread thread = new Thread(task);
```

You can then invoke the **start()** method to tell the JVM that the thread is ready to run, as follows:

```
thread.start();
```

The JVM will execute the task by invoking the task's **run()** method. Figure 32.2b outlines the major steps for creating a task, a thread, and starting the thread.

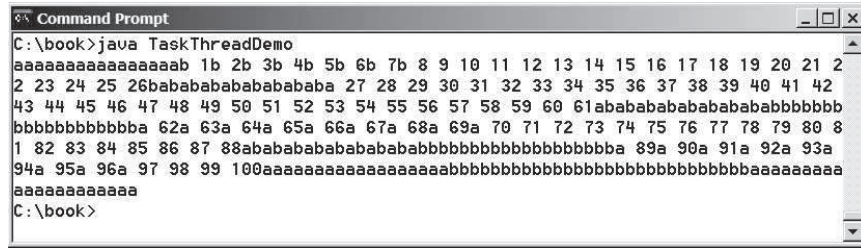
Listing 32.1 gives a program that creates three tasks and three threads to run them.

- The first task prints the letter *a* 100 times.
- The second task prints the letter *b* 100 times.
- The third task prints the integers 1 through 100.

When you run this program, the three threads will share the CPU and take turns printing letters and numbers on the console. Figure 32.3 shows a sample run of the program.

### LISTING 32.1 TaskThreadDemo.java

```
1 public class TaskThreadDemo {
2     public static void main(String[] args) {
3         // Create tasks
4         Runnable printA = new PrintChar('a', 100);
5         Runnable printB = new PrintChar('b', 100);
6         Runnable print100 = new PrintNum(100);
```



**FIGURE 32.3** Tasks `printA`, `printB`, and `print100` are executed simultaneously to display the letter **a** 100 times, the letter **b** 100 times, and the numbers from 1 to 100.

```

7      // Create threads
8      Thread thread1 = new Thread(printA);
9      Thread thread2 = new Thread(printB);
10     Thread thread3 = new Thread(print100);
11
12     // Start threads
13     thread1.start();
14     thread2.start();
15     thread3.start();
16 }
17 }
18
19 // The task for printing a character a specified number of times
20 class PrintChar implements Runnable {
21     private char charToPrint; // The character to print
22     private int times; // The number of times to repeat
23
24     /** Construct a task with specified character and number of
25      * times to print the character
26      */
27     public PrintChar(char c, int t) {
28         charToPrint = c;
29         times = t;
30     }
31
32     @Override /** Override the run() method to tell the system
33      * what task to perform
34      */
35     public void run() {
36         for (int i = 0; i < times; i++) {
37             System.out.print(charToPrint);
38         }
39     }
40 }
41
42 // The task class for printing numbers from 1 to n for a given n
43 class PrintNum implements Runnable {
44     private int lastNum;
45
46     /** Construct a task for printing 1, 2, ..., n */
47     public PrintNum(int n) {
48         lastNum = n;
49     }
50
51     @Override /** Tell the thread how to run */
52     public void run() {
53         for (int i = 1; i <= lastNum; i++) {

```

```

55         System.out.print(" " + i);
56     }
57 }
58 }

```

The program creates three tasks (lines 4–6). To run them concurrently, three threads are created (lines 9–11). The `start()` method (lines 14–16) is invoked to start a thread that causes the `run()` method in the task to be executed. When the `run()` method completes, the thread terminates.

Because the first two tasks, `printA` and `printB`, have similar functionality, they can be defined in one task class `PrintChar` (lines 21–41). The `PrintChar` class implements `Runnable` and overrides the `run()` method (lines 36–40) with the print-character action. This class provides a framework for printing any single character a given number of times. The runnable objects `printA` and `printB` are instances of the `PrintChar` class.

The `PrintNum` class (lines 44–58) implements `Runnable` and overrides the `run()` method (lines 53–57) with the print-number action. This class provides a framework for printing numbers from 1 to  $n$ , for any integer  $n$ . The runnable object `print100` is an instance of the class `PrintNum` class.



### Note

If you don't see the effect of these three threads running concurrently, increase the number of characters to be printed. For example, change line 4 to

```
Runnable printA = new PrintChar('a', 10000);
```

effect of concurrency



### Important Note

The `run()` method in a task specifies how to perform the task. This method is automatically invoked by the JVM. You should not invoke it. Invoking `run()` directly merely executes this method in the same thread; no new thread is started.

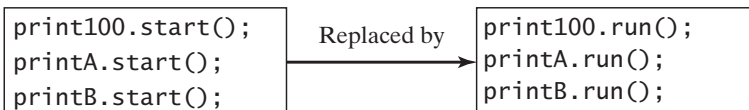
`run()` method

**32.3** How do you define a task class? How do you create a thread for a task?

**32.4** What would happen if you replaced the `start()` method with the `run()` method in lines 14–16 in Listing 32.1?



MyProgrammingLab™



**32.5** What is wrong in the following two programs? Correct the errors.

```

public class Test implements Runnable {
    public static void main(String[] args) {
        new Test();
    }

    public Test() {
        Test task = new Test();
        new Thread(task).start();
    }

    public void run() {
        System.out.println("test");
    }
}

```

(a)

```

public class Test implements Runnable {
    public static void main(String[] args) {
        new Test();
    }

    public Test() {
        Thread t = new Thread(this);
        t.start();
        t.start();
    }

    public void run() {
        System.out.println("test");
    }
}

```

(b)

### 32.4 The Thread Class



The **Thread** class contains the constructors for creating threads for tasks and the methods for controlling threads.

Figure 32.4 shows the class diagram for the **Thread** class.

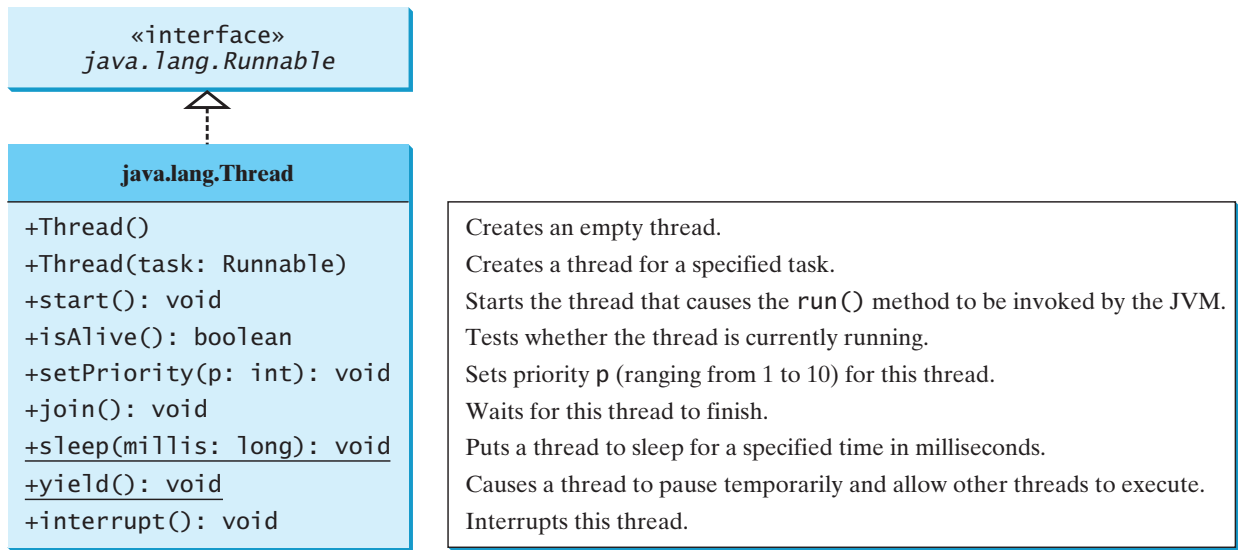


FIGURE 32.4 The **Thread** class contains the methods for controlling threads.



**Note** Since the **Thread** class implements **Runnable**, you could define a class that extends **Thread** and implements the `run` method, as shown in Figure 32.5a, and then create an object from the class and invoke its `start` method in a client program to start the thread, as shown in Figure 32.5b.

separating task from thread

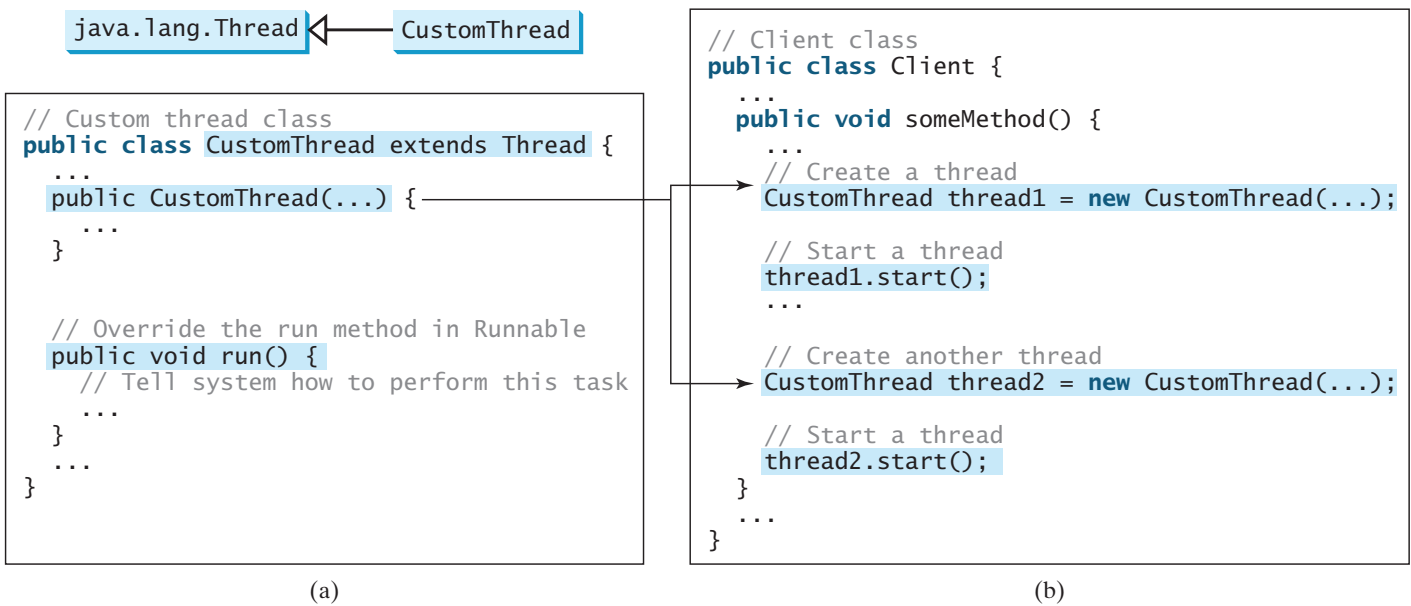


FIGURE 32.5 Define a thread class by extending the **Thread** class.

This approach is, however, not recommended, because it mixes the task and the mechanism of running the task. Separating the task from the thread is a preferred design.



### Note

The **Thread** class also contains the **stop()**, **suspend()**, and **resume()** methods. As of Java 2, these methods were *deprecated* (or *outdated*) because they are known to be inherently unsafe. Instead of using the **stop()** method, you should assign **null** to a **Thread** variable to indicate that it is stopped.

deprecated method

You can use the **yield()** method to temporarily release time for other threads. For example, suppose you modify the code in the **run()** method in lines 53–57 for **PrintNum** in Listing 32.1 as follows:

**yield()**

```
public void run() {
    for (int i = 1; i <= lastNum; i++) {
        System.out.print(" " + i);
        Thread.yield();
    }
}
```

Every time a number is printed, the thread of the **print100** task is yielded to other threads.

The **sleep(long millis)** method puts the thread to sleep for the specified time in milliseconds to allow other threads to execute. For example, suppose you modify the code in lines 53–57 in Listing 32.1 as follows:

**sleep(long)**

```
public void run() {
    try {
        for (int i = 1; i <= lastNum; i++) {
            System.out.print(" " + i);
            if (i >= 50) Thread.sleep(1);
        }
    } catch (InterruptedException ex) {
    }
}
```

Every time a number (**>= 50**) is printed, the thread of the **print100** task is put to sleep for 1 millisecond.

The **sleep** method may throw an **InterruptedException**, which is a checked exception. Such an exception may occur when a sleeping thread's **interrupt()** method is called. The **interrupt()** method is very rarely invoked on a thread, so an **InterruptedException** is unlikely to occur. But since Java forces you to catch checked exceptions, you have to put it in a **try-catch** block. If a **sleep** method is invoked in a loop, you should wrap the loop in a **try-catch** block, as shown in (a) below. If the loop is outside the **try-catch** block, as shown in (b), the thread may continue to execute even though it is being interrupted.

**InterruptedException**

```
public void run() {
    try {
        while (...) {
            ...
            Thread.sleep(1000);
        }
    } catch (InterruptedException ex) {
        ex.printStackTrace();
    }
}
```

(a) Correct

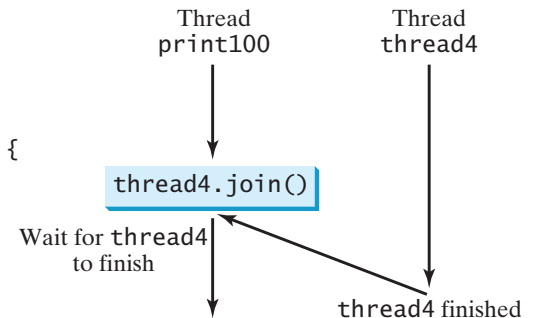
```
public void run() {
    while (...) {
        try {
            ...
            Thread.sleep(sleepTime);
        }
        catch (InterruptedException ex) {
            ex.printStackTrace();
        }
    }
}
```

(b) Incorrect

join()

You can use the `join()` method to force one thread to wait for another thread to finish. For example, suppose you modify the code in lines 53–57 in Listing 32.1 as follows:

```
public void run() {
    Thread thread4 = new Thread(
        new PrintChar('c', 40));
    thread4.start();
    try {
        for (int i = 1; i <= lastNum; i++) {
            System.out.print(" " + i);
            if (i == 50) thread4.join();
        }
    } catch (InterruptedException ex) {
    }
}
```



A new `thread4` is created, and it prints character `c` 40 times. The numbers from 50 to 100 are printed after thread `thread4` is finished.

setPriority(int)

Java assigns every thread a priority. By default, a thread inherits the priority of the thread that spawned it. You can increase or decrease the priority of any thread by using the `setPriority` method, and you can get the thread's priority by using the `getPriority` method. Priorities are numbers ranging from 1 to 10. The `Thread` class has the `int` constants `MIN_PRIORITY`, `NORM_PRIORITY`, and `MAX_PRIORITY`, representing 1, 5, and 10, respectively. The priority of the main thread is `Thread.NORM_PRIORITY`.

round-robin scheduling

The JVM always picks the currently runnable thread with the highest priority. A lower-priority thread can run only when no higher-priority threads are running. If all runnable threads have equal priorities, each is assigned an equal portion of the CPU time in a circular queue. This is called *round-robin scheduling*. For example, suppose you insert the following code in line 16 in Listing 32.1:

```
thread3.setPriority(Thread.MAX_PRIORITY);
```

The thread for the `print100` task will be finished first.



### Tip

The priority numbers may be changed in a future version of Java. To minimize the impact of any changes, use the constants in the `Thread` class to specify thread priorities.



### Tip

A thread may never get a chance to run if there is always a higher-priority thread running or a same-priority thread that never yields. This situation is known as *contention* or *starvation*. To avoid contention, the thread with higher priority must periodically invoke the `sleep` or `yield` method to give a thread with a lower or the same priority a chance to run.

contention or starvation



MyProgrammingLab™

**32.6** Which of the following methods are instance methods in `java.lang.Thread`? Which method may throw an `InterruptedException`? Which of them are deprecated in Java?

`run`, `start`, `stop`, `suspend`, `resume`, `sleep`, `interrupt`, `yield`, `join`

**32.7** If a loop contains a method that throws an `InterruptedException`, why should the loop be placed inside a `try-catch` block?

**32.8** How do you set a priority for a thread? What is the default priority?

## 32.5 Case Study: Flashing Text

*You can use a thread to control an animation.*



The use of a **Timer** object to control animations was introduced in Section 16.11, Animation Using the **Timer** Class. You can also use a thread to control animation. Listing 32.2 gives an example that displays flashing text on a label, as shown in Figure 32.6.



**FIGURE 32.6** The text “Welcome” blinks.

### LISTING 32.2 FlashingText.java

```

1  import javax.swing.*;
2
3  public class FlashingText extends JApplet implements Runnable {
4      private JLabel jlblText = new JLabel("Welcome", JLabel.CENTER);
5
6      public FlashingText() {
7          add(jlblText);
8          new Thread(this).start();
9      }
10
11     @Override /** Set the text on/off every 200 milliseconds */
12     public void run() {
13         try {
14             while (true) {
15                 if (jlblText.getText() == null)
16                     jlblText.setText("Welcome");
17                 else
18                     jlblText.setText(null);
19
20                 Thread.sleep(200);
21             }
22         }
23         catch (InterruptedException ex) {
24         }
25     }
26 }
```

implements Runnable  
create a label

add a label  
start a thread

how to run

sleep

main method omitted

**FlashingText** implements **Runnable** (line 3), so it is a task class. Line 8 wraps the task in a thread and starts the thread. The **run** method dictates how to run the thread. It sets a text in the label if the label does not have one (line 15), and sets its text as **null** (line 18) if the label has a text. The text is set and unset to simulate a flashing effect.

You can use a timer or a thread to control animation. Which one is better? A timer is a source component that fires an **ActionEvent** at a “fixed rate.” When an action event occurs, the timer invokes the listener’s **actionPerformed** method to handle the event. The timer and event handling run on the same thread. If it takes a long time to handle the event, the actual delay time between the two events will be longer than the requested delay time. In this case, you should run event handling on a separate thread. (The next section gives an example to illustrate the problem and how to fix it by running the event handling on a separate thread.) In general, threads are more reliable and responsive than timers. If you need a precise delay

thread vs. timer

time or a quick response, it is better to use a thread. Otherwise, using a timer is simpler and more efficient. Timers consume less system resources because they run on the GUI event dispatch thread, so you don't need to spawn new threads for timers.



**32.9** What causes the text to flash?

**32.10** Is an instance of **FlashingText** a runnable object?

MyProgrammingLab™



## 32.6 GUI Event Dispatch Thread

*GUI event handling code is executed on a special thread called the event dispatch thread.*

event dispatch thread

This special thread is also used to run most of Swing methods. Running GUI event handling code and the most of Swing methods in the same thread is necessary because most Swing methods are not thread-safe. Invoking them from multiple threads may cause conflicts.

In certain situations, you need to run the code in the event dispatch thread to avoid possible conflicts. You can use the static methods **invokeLater** and **invokeAndWait** in the **javax.swing.SwingUtilities** class to run the code in the event dispatch thread. You must put this code in the **run** method of a **Runnable** object and specify the **Runnable** object as the argument to **invokeLater** and **invokeAndWait**. The **invokeLater** method returns immediately, without waiting for the event dispatch thread to execute the code. The **invokeAndWait** method is just like **invokeLater**, except that **invokeAndWait** doesn't return until the event dispatching thread has executed the specified code.

invokeLater  
invokeAndWait

So far, you have launched your GUI application from the **main** method by creating a frame and making it visible. This works fine for most applications, but if it takes a long time to launch a GUI application, problems may occur. To avoid possible problems in this situation, you should launch the GUI creation from the event dispatch thread, as follows:

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            // The code for creating a frame and setting its properties
        }
    });
}
```

For example, Listing 32.3 gives a simple program that launches the frame from the event dispatch thread.

### LISTING 32.3 EventDispatcherThreadDemo.java

```
1  import javax.swing.*;
2
3  public class EventDispatcherThreadDemo extends JApplet {
4      public EventDispatcherThreadDemo() {
5          add(new JLabel("Hi, it runs from an event dispatch thread"));
6      }
7
8      /** Main method */
9      public static void main(String[] args) {
10         SwingUtilities.invokeLater(new Runnable() {
11             public void run() {
12                 JFrame frame = new JFrame("EventDispatcherThreadDemo");
13                 frame.add(new EventDispatcherThreadDemo());
14                 frame.setSize(200, 200);
15                 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16                 frame.setLocationRelativeTo(null); // Center the frame
```

create frame

```

17         frame.setVisible(true);
18     }
19     });
20 }
21 }

```

**32.11** What is the event dispatch thread?

**32.12** How do you let a task run from the event dispatch thread?



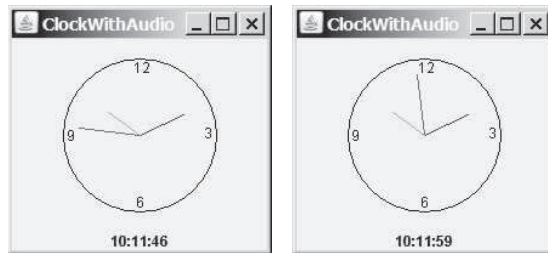
MyProgrammingLab™

## 32.7 Case Study: Clock with Audio

*This case study shows the necessity of using threads for certain GUI animations.*



This case study creates an applet that displays a running clock that announces the time at one-minute intervals. For example, if the current time is 6:30:00, the applet announces, “six o’clock thirty minutes A.M.” If the current time is 20:20:00, the applet announces, “eight o’clock twenty minutes P.M.” The program also has a label that displays the digital time, as shown in Figure 32.7.



**FIGURE 32.7** The applet displays a clock and announces the time every minute.

To announce the time, the applet plays three audio clips. The first clip announces the hour, the second announces the minute, and the third announces A.M. or P.M. All of the audio files are stored in the directory **audio**, a subdirectory of the applet’s class directory. The 12 audio files used to announce the hours are stored in the files **hour0.au**, **hour1.au**, and so on, to **hour11.au**. The 60 audio files used to announce the minutes are stored in the files **minute0.au**, **minute1.au**, and so on, to **minute59.au**. The two audio files used to announce A.M. or P.M. are stored in the file **am.au** and **pm.au**.

audio clips  
audio files

You need to play three audio clips on a separate thread to avoid animation delays. To illustrate the problem, let us first write a program without playing the audio on a separate thread.

In Section 13.9, the **StillClock** class was developed to draw a still clock to show the current time. Create an applet named **ClockWithAudio** (Listing 32.4) that contains an instance of **StillClock** to display an analog clock, and an instance of **JLabel** to display the digital time. Override the **init** method to load the audio files. Use a **Timer** object to set and display the current time continuously at every second. When the second is zero, announce the current time.

### LISTING 32.4 ClockWithAudio.java

```

1  import java.applet.*;
2  import javax.swing.*;
3  import java.awt.event.*;
4  import java.awt.*;
5
6  public class ClockWithAudio extends JApplet {
7      protected AudioClip[] hourAudio = new AudioClip[12];

```

audio clips

```

8      protected AudioClip[] minuteAudio = new AudioClip[60];
9
10     // Create audio clips for pronouncing am and pm
11     protected AudioClip amAudio =
12         Applet.newAudioClip(this.getClass().getResource("audio/am.au"));
13     protected AudioClip pmAudio =
14         Applet.newAudioClip(this.getClass().getResource("audio/pm.au"));
15
16     // Create a clock
17     private StillClock clock = new StillClock();
18
19     // Create a timer
20     private Timer timer = new Timer(1000, new TimerListener());
21
22     // Create a label to display time
23     private JLabel jlblDigitTime = new JLabel("", JLabel.CENTER);
24
25     @Override /** Initialize the applet */
26     public void init() {
27         // Create audio clips for pronouncing hours
28         for (int i = 0; i < 12; i++)
29             hourAudio[i] = Applet.newAudioClip(
30                 this.getClass().getResource("audio/hour" + i + ".au"));
31
32         // Create audio clips for pronouncing minutes
33         for (int i = 0; i < 60; i++)
34             minuteAudio[i] = Applet.newAudioClip(
35                 this.getClass().getResource("audio/minute" + i + ".au"));
36
37         // Add clock and time label to the content pane of the applet
38         add(clock, BorderLayout.CENTER);
39         add(jlblDigitTime, BorderLayout.SOUTH);
40     }
41
42     @Override /** Override the applet's start method */
43     public void start() {
44         timer.start(); // Resume clock
45     }
46
47     @Override /** Override the applet's stop method */
48     public void stop() {
49         timer.stop(); // Suspend clock
50     }
51
52     private class TimerListener implements ActionListener {
53         @Override
54         public void actionPerformed(ActionEvent e) {
55             clock.setCurrentTime();
56             clock.repaint();
57             jlblDigitTime.setText(clock.getHour() + ":" + clock.getMinute()
58                 + ":" + clock.getSecond());
59             if (clock.getSecond() == 0)
60                 announceTime(clock.getHour(), clock.getMinute());
61         }
62     }
63
64     /** Announce the current time at every minute */
65     public void announceTime(int hour, int minute) {
66         // Announce hour
67         hourAudio[hour % 12].play();

```

am clip

pm clip

still clock

timer

label

create audio clips

start timer

stop timer

timer listener

set new time

announce time

announce hour

```

68
69     try {
70         // Time delay to allow hourAudio play to finish
71         Thread.sleep(1500);
72
73         // Announce minute
74         minuteAudio[minute].play();
75
76         // Time delay to allow minuteAudio play to finish
77         Thread.sleep(1500);
78     }
79     catch (InterruptedException ex) {
80     }
81
82     // Announce am or pm
83     if (hour < 12)
84         amAudio.play();
85     else
86         pmAudio.play();
87 }
88 }

```

announce minute  
announce am  
announce pm  
main method omitted

The `hourAudio` is an array of twelve audio clips that are used to announce the 12 hours of the day (line 7); the `minuteAudio` is an audio clip that is used to announce the minutes in an hour (line 8). The `amAudio` announces “A.M.” (line 11); the `pmAudio` announces “P.M.” (line 13).

The `init()` method creates hour audio clips (lines 29–30) and minute audio clips (lines 34–35), and places a clock and a label in the applet (lines 38–39).

An `ActionEvent` is fired by the timer every second. In the listener’s `actionPerformed` method (lines 54–61), the clock is repainted with the new current time, and the digital time is displayed in the label.

In the `announceTime` method (lines 65–87), the `sleep()` method (lines 71, 77) is purposely invoked to ensure that one clip finishes before the next clip starts, so that the clips do not interfere with each other.

The applet’s `start()` and `stop()` methods (lines 43–50) are overridden to ensure that the timer starts or stops when the applet is restarted or stopped.

When you run the preceding program, you will notice that the second hand does not display at the first, second, and third seconds of the minute. This is because `sleep(1500)` is invoked twice in the `announceTime()` method, which takes three seconds to announce the time at the beginning of each minute. Thus, the next action event is delayed for three seconds during the first three seconds of each minute. As a result of this delay, the time is not updated and the clock is not repainted for these three seconds. To fix this problem, you should announce the time on a separate thread. This can be accomplished by modifying the `announceTime` method. Listing 32.5 gives the new program.

abnormal problem

### LISTING 32.5 ClockWithAudioOnSeparateThread.java

```

1  // Same import statements as in Listing 32.4, so omitted
2
3  public class ClockWithAudioOnSeparateThread extends JApplet {
4      // Same as in lines 7–62 in Listing 32.4, so omitted
5
6      /** Announce the current time at every minute */
7      public void announceTime(int h, int m) {
8          new Thread(new AnnounceTimeOnSeparateThread(h, m)).start();
9      }

```

omitted  
omitted  
create a thread

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| task class          | <pre> 10 11 /** Inner class for announcing time */ 12 class AnnounceTimeOnSeparateThread implements Runnable { 13     private int hour, minute; 14 15     /** Get audio clips */ 16     public AnnounceTimeOnSeparateThread(int hour, int minute) { 17         this.hour = hour; 18         this.minute = minute; 19     } 20 21     @Override 21     public void run() { 22         // Announce hour 23         hourAudio[hour % 12].play(); 24 25         try { 26             // Time delay to allow hourAudio play to finish 27             Thread.sleep(1500); 28 29             // Announce minute 30             minuteAudio[minute].play(); 31 32             // Time delay to allow minuteAudio play to finish 33             Thread.sleep(1500); 34         } 35         catch (InterruptedException ex) { 36         } 37 38         // Announce am or pm 39         if (hour &lt; 12) 40             amAudio.play(); 41         else 42             pmAudio.play(); 43     } 44 } </pre> |
| run thread          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| main method omitted | <pre> 45 } </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

The new class `ClockWithAudioOnSeparateThread` is the same as `ClockWithAudio` except that the `announceTime` method is new. The new `announceTime` method creates a thread (line 8) for the task of announcing the time. The task class is defined as an inner class (lines 12–44). The `run` method (line 21) announces the time on a separate thread.

When running this program, you will discover that the audio does not interfere with the clock animation because an instance of `AnnounceTimeOnSeparateThread` starts on a separate thread to announce the current time. This thread is independent of the thread on which the `actionPerformed` method runs.



**32.13** When should you use a timer or a thread to control animation? What are the advantages and disadvantages of using a thread and a timer?

MyProgrammingLab™



## 32.8 Thread Pools

*A thread pool can be used to execute tasks efficiently.*

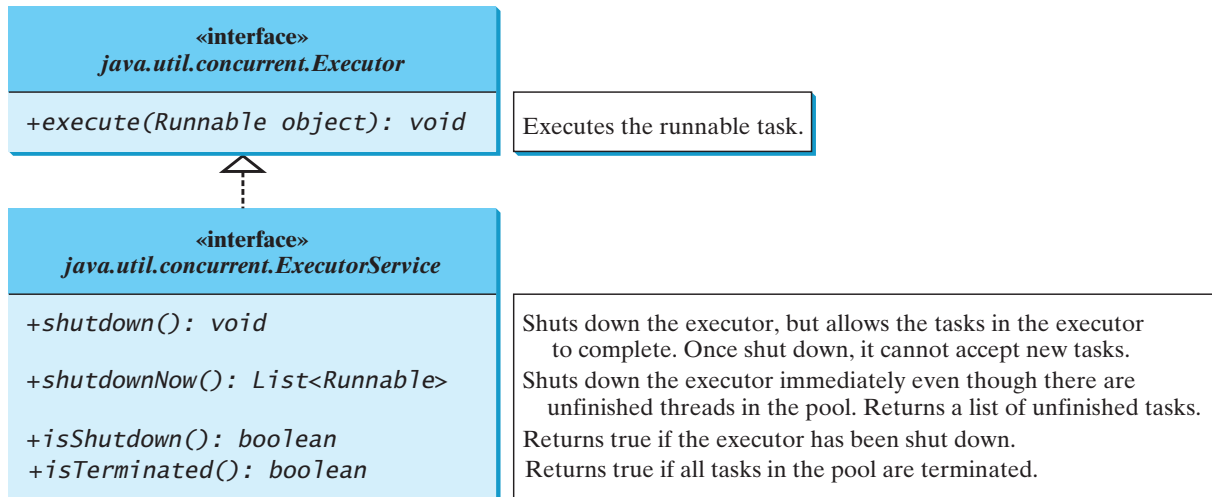
In Section 32.3, Creating Tasks and Threads, you learned how to define a task class by implementing `java.lang.Runnable`, and how to create a thread to run a task like this:

```

Runnable task = new TaskClass(task);
new Thread(task).start();

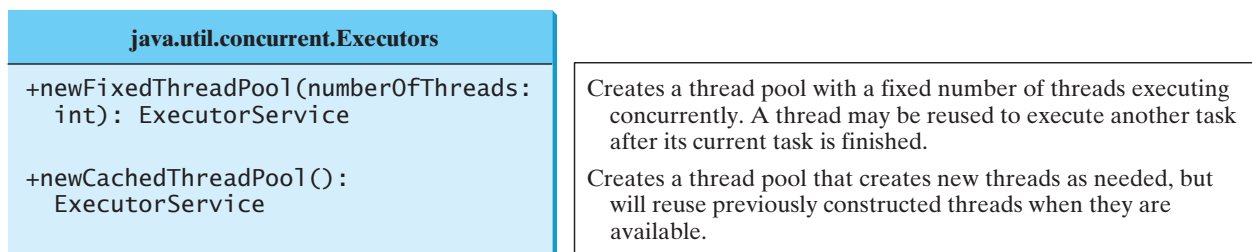
```

This approach is convenient for a single task execution, but it is not efficient for a large number of tasks, because you have to create a thread for each task. Starting a new thread for each task could limit throughput and cause poor performance. Using a *thread pool* is an ideal way to manage the number of tasks executing concurrently. Java provides the **Executor** interface for executing tasks in a thread pool and the **ExecutorService** interface for managing and controlling tasks. **ExecutorService** is a subinterface of **Executor**, as shown in Figure 32.8.



**FIGURE 32.8** The **Executor** interface executes threads, and the **ExecutorService** subinterface manages threads.

To create an **Executor** object, use the static methods in the **Executors** class, as shown in Figure 32.9. The `newFixedThreadPool(int)` method creates a fixed number of threads in a pool. If a thread completes executing a task, it can be reused to execute another task. If a thread terminates due to a failure prior to shutdown, a new thread will be created to replace it if all the threads in the pool are not idle and there are tasks waiting for execution. The `newCachedThreadPool()` method creates a new thread if all the threads in the pool are not idle and there are tasks waiting for execution. A thread in a cached pool will be terminated if it has not been used for 60 seconds. A cached pool is efficient for many short tasks.



**FIGURE 32.9** The **Executors** class provides static methods for creating **Executor** objects.

Listing 32.6 shows how to rewrite Listing 32.1 using a thread pool.

### LISTING 32.6 ExecutorDemo.java

```

1 import java.util.concurrent.*;
2
3 public class ExecutorDemo {

```

```

4    public static void main(String[] args) {
5        // Create a fixed thread pool with maximum three threads
6        ExecutorService executor = Executors.newFixedThreadPool(3);
7
8        // Submit runnable tasks to the executor
9        executor.execute(new PrintChar('a', 100));
10       executor.execute(new PrintChar('b', 100));
11       executor.execute(new PrintNum(100));
12
13       // Shut down the executor
14       executor.shutdown();
15   }
16 }

```

create executor

submit task

shut down executor

Line 6 creates a thread pool executor with a total of three threads maximum. Classes `PrintChar` and `PrintNum` were defined in Listing 32.1. Line 9 creates a task, `new PrintChar('a', 100)`, and adds it to the pool. Similarly, another two runnable tasks are created and added to the same pool in lines 10–11. The executor creates three threads to execute three tasks concurrently.

Suppose that you replace line 6 with

```
ExecutorService executor = Executors.newFixedThreadPool(1);
```

What will happen? The three runnable tasks will be executed sequentially, because there is only one thread in the pool.

Suppose you replace line 6 with

```
ExecutorService executor = Executors.newCachedThreadPool();
```

What will happen? New threads will be created for each waiting task, so all the tasks will be executed concurrently.

The `shutdown()` method in line 14 tells the executor to shut down. No new tasks can be accepted, but any existing tasks will continue to finish.



### Tip

If you need to create a thread for just one task, use the `Thread` class. If you need to create threads for multiple tasks, it is better to use a thread pool.



**32.14** What are the benefits of using a thread pool?

**32.15** How do you create a thread pool with three fixed threads? How do you submit a task to a thread pool? How do you know that all the tasks are finished?

MyProgrammingLab™

## 32.9 Thread Synchronization



*Thread synchronization is to coordinate the execution of the dependent threads.*

A shared resource may become corrupted if it is accessed simultaneously by multiple threads. The following example demonstrates the problem.

Suppose that you create and launch 100 threads, each of which adds a penny to an account. Define a class named `Account` to model the account, a class named `AddAPennyTask` to add a penny to the account, and a main class that creates and launches threads. The relationships of these classes are shown in Figure 32.10. The program is given in Listing 32.7.

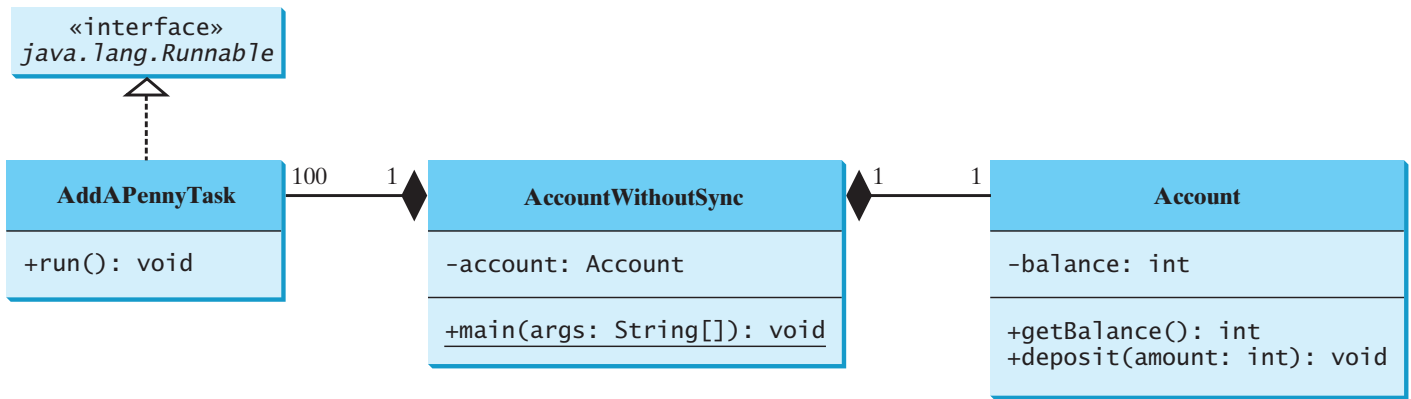


FIGURE 32.10 **AccountWithoutSync** contains an instance of **Account** and 100 threads of **AddAPennyTask**.

### LISTING 32.7 AccountWithoutSync.java

```

1  import java.util.concurrent.*;
2
3  public class AccountWithoutSync {
4      private static Account account = new Account();
5
6      public static void main(String[] args) {
7          ExecutorService executor = Executors.newCachedThreadPool();           create executor
8
9          // Create and launch 100 threads
10         for (int i = 0; i < 100; i++) {
11             executor.execute(new AddAPennyTask());                           submit task
12         }
13
14         executor.shutdown();                                                  shut down executor
15
16         // Wait until all tasks are finished
17         while (!executor.isTerminated()) {                                    wait for all tasks to terminate
18         }
19
20         System.out.println("What is balance? " + account.getBalance());
21     }
22
23     // A thread for adding a penny to the account
24     private static class AddAPennyTask implements Runnable {
25         public void run() {
26             account.deposit(1);
27         }
28     }
29
30     // An inner class for account
31     private static class Account {
32         private int balance = 0;
33
34         public int getBalance() {
35             return balance;
36         }
37
38         public void deposit(int amount) {
39             int newBalance = balance + amount;
40

```

```

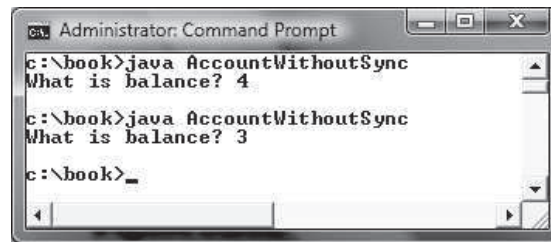
41         // This delay is deliberately added to magnify the
42         // data-corruption problem and make it easy to see.
43         try {
44             Thread.sleep(5);
45         }
46         catch (InterruptedException ex) {
47         }
48
49         balance = newBalance;
50     }
51 }
52 }

```

The classes `AddAPennyTask` and `Account` in lines 24–51 are inner classes. Line 4 creates an `Account` with initial balance 0. Line 11 creates a task to add a penny to the account and submit the task to the executor. Line 11 is repeated 100 times in lines 10–12. The program repeatedly checks whether all tasks are completed in lines 17–18. The account balance is displayed in line 20 after all tasks are completed.

The program creates 100 threads executed in a thread pool `executor` (lines 10–12). The `isTerminated()` method (line 17) is used to test whether the thread is terminated.

The balance of the account is initially 0 (line 32). When all the threads are finished, the balance should be 100, but the output is unpredictable. As can be seen in Figure 32.11, the answers are wrong in the sample run. This demonstrates the data-corruption problem that occurs when all the threads have access to the same data source simultaneously.



**FIGURE 32.11** The `AccountWithoutSync` program causes data inconsistency.

Lines 39–49 could be replaced by one statement:

```
balance = balance + amount;
```

It is highly unlikely, although plausible, that the problem can be replicated using this single statement. The statements in lines 39–49 are deliberately designed to magnify the data-corruption problem and make it easy to see. If you run the program several times but still do not see the problem, increase the sleep time in line 44. This will increase the chances for showing the problem of data inconsistency.

What, then, caused the error in this program? A possible scenario is shown in Figure 32.12.

| Step | Balance | Task 1                                 | Task 2                                 |
|------|---------|----------------------------------------|----------------------------------------|
| 1    | 0       | <code>newBalance = balance + 1;</code> |                                        |
| 2    | 0       |                                        | <code>newBalance = balance + 1;</code> |
| 3    | 1       | <code>balance = newBalance;</code>     |                                        |
| 4    | 1       |                                        | <code>balance = newBalance;</code>     |

**FIGURE 32.12** Task 1 and Task 2 both add 1 to the same balance.

In Step 1, Task 1 gets the balance from the account. In Step 2, Task 2 gets the same balance from the account. In Step 3, Task 1 writes a new balance to the account. In Step 4, Task 2 writes a new balance to the account.

The effect of this scenario is that Task 1 does nothing, because in Step 4 Task 2 overrides Task 1's result. Obviously, the problem is that Task 1 and Task 2 are accessing a common resource in a way that causes a conflict. This is a common problem, known as a *race condition*, in multithreaded programs. A class is said to be *thread-safe* if an object of the class does not cause a race condition in the presence of multiple threads. As demonstrated in the preceding example, the `Account` class is not thread-safe.

race condition  
thread-safe

### 32.9.1 The `synchronized` Keyword

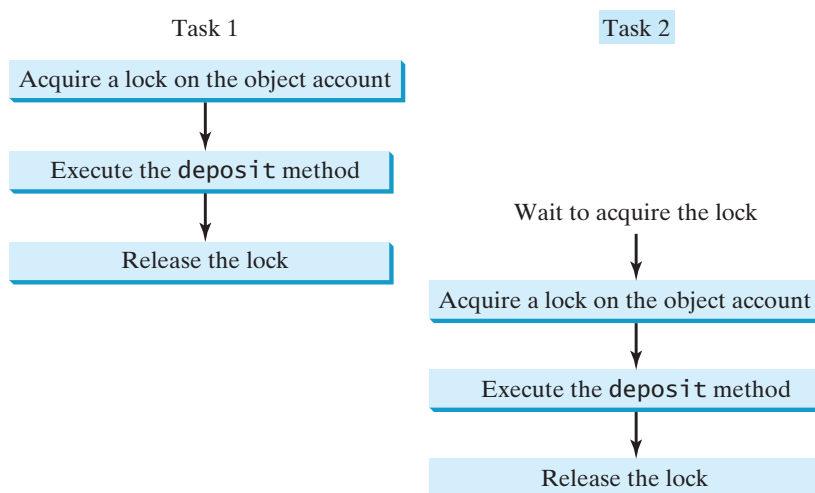
To avoid race conditions, it is necessary to prevent more than one thread from simultaneously entering a certain part of the program, known as the *critical region*. The critical region in Listing 32.7 is the entire `deposit` method. You can use the keyword `synchronized` to synchronize the method so that only one thread can access the method at a time. There are several ways to correct the problem in Listing 32.7. One approach is to make `Account` thread-safe by adding the keyword `synchronized` in the `deposit` method in line 38, as follows:

critical region

```
public synchronized void deposit(double amount)
```

A synchronized method acquires a lock before it executes. A lock is a mechanism for exclusive use of a resource. In the case of an instance method, the lock is on the object for which the method was invoked. In the case of a static method, the lock is on the class. If one thread invokes a synchronized instance method (respectively, static method) on an object, the lock of that object (respectively, class) is acquired first, then the method is executed, and finally the lock is released. Another thread invoking the same method of that object (respectively, class) is blocked until the lock is released.

With the `deposit` method synchronized, the preceding scenario cannot happen. If Task 1 enters the method, Task 2 is blocked until Task 1 finishes the method, as shown in Figure 32.13.



**FIGURE 32.13** Task 1 and Task 2 are synchronized.

### 32.9.2 Synchronizing Statements

Invoking a synchronized instance method of an object acquires a lock on the object, and invoking a synchronized static method of a class acquires a lock on the class. A synchronized statement can be used to acquire a lock on any object, not just *this* object, when executing a

synchronized block

block of the code in a method. This block is referred to as a *synchronized block*. The general form of a synchronized statement is as follows:

```
synchronized (expr) {
    statements;
}
```

The expression **expr** must evaluate to an object reference. If the object is already locked by another thread, the thread is blocked until the lock is released. When a lock is obtained on the object, the statements in the synchronized block are executed, and then the lock is released.

Synchronized statements enable you to synchronize part of the code in a method instead of the entire method. This increases concurrency. You can make Listing 32.7 thread-safe by placing the statement in line 26 inside a synchronized block:

```
synchronized (account) {
    account.deposit(1);
}
```



### Note

Any synchronized instance method can be converted into a synchronized statement. For example, the following synchronized instance method in (a) is equivalent to (b):

```
public synchronized void xMethod() {
    // method body
}
```

(a)

```
public void xMethod() {
    synchronized (this) {
        // method body
    }
}
```

(b)



Check  
Point

MyProgrammingLab™

**32.16** Give some examples of possible resource corruption when running multiple threads. How do you synchronize conflicting threads?

**32.17** Suppose you place the statement in line 26 of Listing 32.7 inside a synchronized block to avoid race conditions, as follows:

```
synchronized (this) {
    account.deposit(1);
}
```

Will it work?



Key  
Point

## 32.10 Synchronization Using Locks

*Locks and conditions can be explicitly used to synchronize threads.*

Recall that in Listing 32.7, 100 tasks deposit a penny to the same account concurrently, which causes conflicts. To avoid it, you used the **synchronized** keyword in the **deposit** method, as follows:

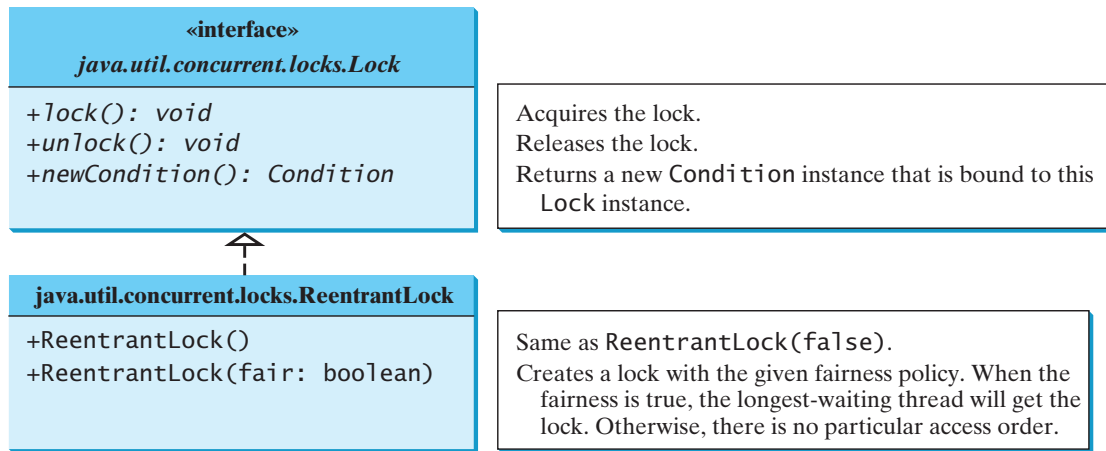
```
public synchronized void deposit(double amount)
```

A synchronized instance method implicitly acquires a lock on the instance before it executes the method.

Java enables you to acquire locks explicitly, which give you more control for coordinating threads. A *lock* is an instance of the **Lock** interface, which defines the methods for

lock

acquiring and releasing locks, as shown in Figure 32.14. A lock may also use the `newCondition()` method to create any number of `Condition` objects, which can be used for thread communications.



**FIGURE 32.14** The `ReentrantLock` class implements the `Lock` interface to represent a lock.

`ReentrantLock` is a concrete implementation of `Lock` for creating mutually exclusive locks. You can create a lock with the specified *fairness policy*. True fairness policies guarantee that the longest-waiting thread will obtain the lock first. False fairness policies grant a lock to a waiting thread arbitrarily. Programs using fair locks accessed by many threads may have poorer overall performance than those using the default setting, but they have smaller variances in times to obtain locks and prevent starvation.

fairness policy

Listing 32.8 revises the program in Listing 32.7 to synchronize the account modification using explicit locks.

### LISTING 32.8 AccountWithSyncUsingLock.java

```

1  import java.util.concurrent.*;
2  import java.util.concurrent.locks.*;
3
4  public class AccountWithSyncUsingLock {
5      private static Account account = new Account();
6
7      public static void main(String[] args) {
8          ExecutorService executor = Executors.newCachedThreadPool();
9
10         // Create and launch 100 threads
11         for (int i = 0; i < 100; i++) {
12             executor.execute(new AddAPennyTask());
13         }
14
15         executor.shutdown();
16
17         // Wait until all tasks are finished
18         while (!executor.isTerminated()) {
19             ;
20         }
21         System.out.println("What is balance? " + account.getBalance());
22     }
23
24     // A thread for adding a penny to the account
  
```

package for locks

```

25     public static class AddAPennyTask implements Runnable {
26         public void run() {
27             account.deposit(1);
28         }
29     }
30
31     // An inner class for Account
32     public static class Account {
33         private static Lock lock = new ReentrantLock(); // Create a lock
34         private int balance = 0;
35
36         public int getBalance() {
37             return balance;
38         }
39
40         public void deposit(int amount) {
41             lock.lock(); // Acquire the lock
42
43             try {
44                 int newBalance = balance + amount;
45
46                 // This delay is deliberately added to magnify the
47                 // data-corruption problem and make it easy to see.
48                 Thread.sleep(5);
49
50                 balance = newBalance;
51             }
52             catch (InterruptedException ex) {
53             }
54             finally {
55                 lock.unlock(); // Release the lock
56             }
57         }
58     }
59 }

```

create a lock

acquire the lock

release the lock

Line 33 creates a lock, line 41 acquires the lock, and line 55 releases the lock.



### Tip

It is a good practice to always immediately follow a call to `lock()` with a `try-catch` block and release the lock in the `finally` clause, as shown in lines 41–56, to ensure that the lock is always released.

Listing 32.8 can be implemented using a `synchronize` method for `deposit` rather than using a lock. In general, using `synchronized` methods or statements is simpler than using explicit locks for mutual exclusion. However, using explicit locks is more intuitive and flexible to synchronize threads with conditions, as you will see in the next section.



**32.18** How do you create a lock object? How do you acquire a lock and release a lock?

MyProgrammingLab™



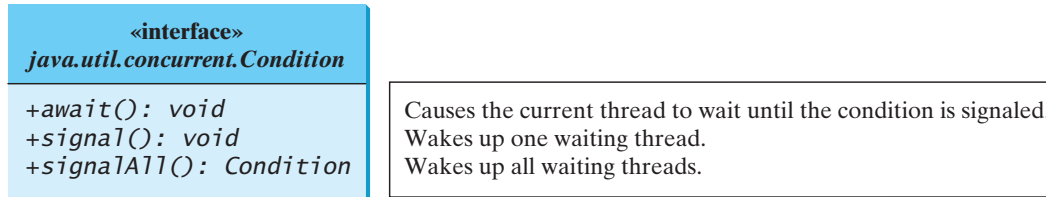
## 32.11 Cooperation among Threads

*Conditions on locks can be used to coordinate thread interactions.*

Thread synchronization suffices to avoid race conditions by ensuring the mutual exclusion of multiple threads in the critical region, but sometimes you also need a way for threads to cooperate. *Conditions* can be used to facilitate communications among threads. A thread can specify what to do under a certain condition. Conditions are objects created by invoking the

condition

`newCondition()` method on a `Lock` object. Once a condition is created, you can use its `await()`, `signal()`, and `signalAll()` methods for thread communications, as shown in Figure 32.15. The `await()` method causes the current thread to wait until the condition is signaled. The `signal()` method wakes up one waiting thread, and the `signalAll()` method wakes all waiting threads.

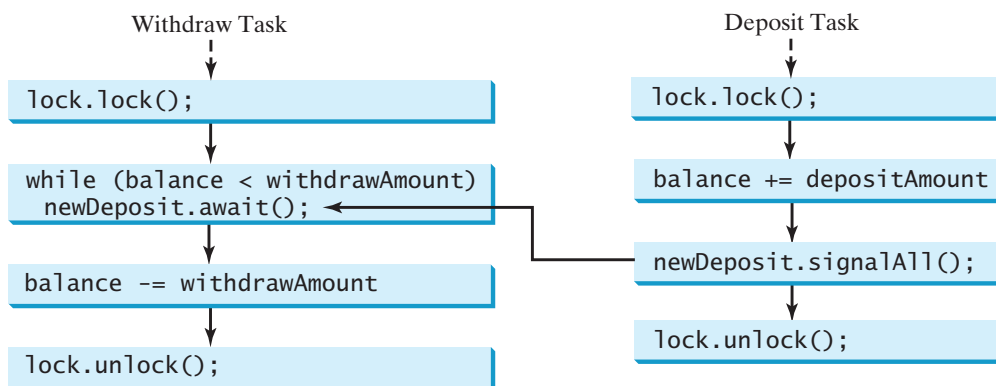


**FIGURE 32.15** The `Condition` interface defines the methods for performing synchronization.

Let us use an example to demonstrate thread communications. Suppose that you create and launch two tasks: one that deposits into an account, and one that withdraws from the same account. The withdraw task has to wait if the amount to be withdrawn is more than the current balance. Whenever new funds are deposited into the account, the deposit task notifies the withdraw thread to resume. If the amount is still not enough for a withdrawal, the withdraw thread has to continue to wait for a new deposit.

thread cooperation example

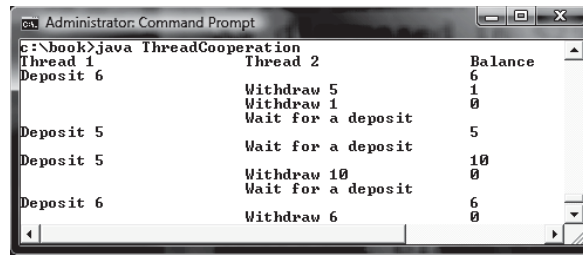
To synchronize the operations, use a lock with a condition: `newDeposit` (i.e., new deposit added to the account). If the balance is less than the amount to be withdrawn, the withdraw task will wait for the `newDeposit` condition. When the deposit task adds money to the account, the task signals the waiting withdraw task to try again. The interaction between the two tasks is shown in Figure 32.16.



**FIGURE 32.16** The condition `newDeposit` is used for communications between the two threads.

You create a condition from a `Lock` object. To use a condition, you have to first obtain a lock. The `await()` method causes the thread to wait and automatically releases the lock on the condition. Once the condition is right, the thread reacquires the lock and continues executing.

Assume that the initial balance is 0 and the amounts to deposit and withdraw are randomly generated. Listing 32.9 gives the program. A sample run of the program is shown in Figure 32.17.



**FIGURE 32.17** The withdraw task waits if there are not sufficient funds to withdraw.

### LISTING 32.9 ThreadCooperation.java

```

1  import java.util.concurrent.*;
2  import java.util.concurrent.locks.*;
3
4  public class ThreadCooperation {
5      private static Account account = new Account();
6
7      public static void main(String[] args) {
8          // Create a thread pool with two threads
9          ExecutorService executor = Executors.newFixedThreadPool(2);
10         executor.execute(new DepositTask());
11         executor.execute(new WithdrawTask());
12         executor.shutdown();
13
14         System.out.println("Thread 1\t\tThread 2\t\tBalance");
15     }
16
17     public static class DepositTask implements Runnable {
18         @Override // Keep adding an amount to the account
19         public void run() {
20             try { // Purposely delay it to let the withdraw method proceed
21                 while (true) {
22                     account.deposit(((int)(Math.random() * 10) + 1));
23                     Thread.sleep(1000);
24                 }
25             }
26             catch (InterruptedException ex) {
27                 ex.printStackTrace();
28             }
29         }
30     }
31
32     public static class WithdrawTask implements Runnable {
33         @Override // Keep subtracting an amount from the account
34         public void run() {
35             while (true) {
36                 account.withdraw(((int)(Math.random() * 10) + 1));
37             }
38         }
39     }
40
41     // An inner class for account
42     private static class Account {
43         // Create a new lock
44         private static Lock lock = new ReentrantLock();
45
46         // Create a condition
47         private static Condition newDeposit = lock.newCondition();

```

create two threads

create a lock

create a condition

```

48
49     private int balance = 0;
50
51     public int getBalance() {
52         return balance;
53     }
54
55     public void withdraw(int amount) {
56         lock.lock(); // Acquire the lock
57         try {
58             while (balance < amount) {
59                 System.out.println("\t\t\tWait for a deposit");
60                 newDeposit.await();
61             }
62
63             balance -= amount;
64             System.out.println("\t\t\tWithdraw " + amount +
65                 "\t\t" + getBalance());
66         }
67         catch (InterruptedException ex) {
68             ex.printStackTrace();
69         }
70         finally {
71             lock.unlock(); // Release the lock
72         }
73     }
74
75     public void deposit(int amount) {
76         lock.lock(); // Acquire the lock
77         try {
78             balance += amount;
79             System.out.println("Deposit " + amount +
80                 "\t\t\t\t\t" + getBalance());
81
82             // Signal thread waiting on the condition
83             newDeposit.signalAll();
84         }
85         finally {
86             lock.unlock(); // Release the lock
87         }
88     }
89 }
90 }

```

acquire the lock

wait on the condition

release the lock

acquire the lock

signal threads

release the lock

The example creates a new inner class named **Account** to model the account with two methods, **deposit(int)** and **withdraw(int)**, a class named **DepositTask** to add an amount to the balance, a class named **WithdrawTask** to withdraw an amount from the balance, and a main class that creates and launches two threads.

The program creates and submits the deposit task (line 10) and the withdraw task (line 11). The deposit task is purposely put to sleep (line 23) to let the withdraw task run. When there are not enough funds to withdraw, the withdraw task waits (line 59) for notification of the balance change from the deposit task (line 83).

A lock is created in line 44. A condition named **newDeposit** on the lock is created in line 47. A condition is bound to a lock. Before waiting or signaling the condition, a thread must first acquire the lock for the condition. The withdraw task acquires the lock in line 56, waits for the **newDeposit** condition (line 60) when there is not a sufficient amount to withdraw, and releases the lock in line 71. The deposit task acquires the lock in line 76, and signals all waiting threads (line 83) for the **newDeposit** condition after a new deposit is made.

What will happen if you replace the `while` loop in lines 58–61 with the following `if` statement?

```
if (balance < amount) {
    System.out.println("\t\t\tWait for a deposit");
    newDeposit.await();
}
```

The deposit task will notify the withdraw task whenever the balance changes. (`balance < amount`) may still be true when the withdraw task is awakened. Using the `if` statement, the withdraw task may wait forever. Using the loop statement, the withdraw task will have a chance to recheck the condition. Thus you should always test the condition in a loop.



Caution

Once a thread invokes `await()` on a condition, the thread waits for a signal to resume. If you forget to call `signal()` or `signalAll()` on the condition, the thread will wait forever.

ever-waiting threads



Caution

A condition is created from a `Lock` object. To invoke its method (e.g., `await()`, `signal()`, and `signalAll()`), you must first own the lock. If you invoke these methods without acquiring the lock, an `IllegalMonitorStateException` will be thrown.

`IllegalMonitorState-`  
`Exception`

Locks and conditions were introduced in Java 5. Prior to Java 5, thread communications were programmed using the object’s built-in monitors. Locks and conditions are more powerful and flexible than the built-in monitor, so you can ignore this section. *However*, if you are working with legacy Java code, you may encounter Java’s built-in monitor.

Java’s built-in monitor  
monitor

A *monitor* is an object with mutual exclusion and synchronization capabilities. Only one thread at a time can execute a method in the monitor. A thread enters the monitor by acquiring a lock on it and exits by releasing the lock. *Any object can be a monitor*. An object becomes a monitor once a thread locks it. Locking is implemented using the `synchronized` keyword on a method or a block. A thread must acquire a lock before executing a synchronized method or block. A thread can wait in a monitor if the condition is not right for it to continue executing in the monitor. You can invoke the `wait()` method on the monitor object to release the lock so that some other thread can get in the monitor and perhaps change the monitor’s state. When the condition is right, the other thread can invoke the `notify()` or `notifyAll()` method to signal one or all waiting threads to regain the lock and resume execution. The template for invoking these methods is shown in Figure 32.18.

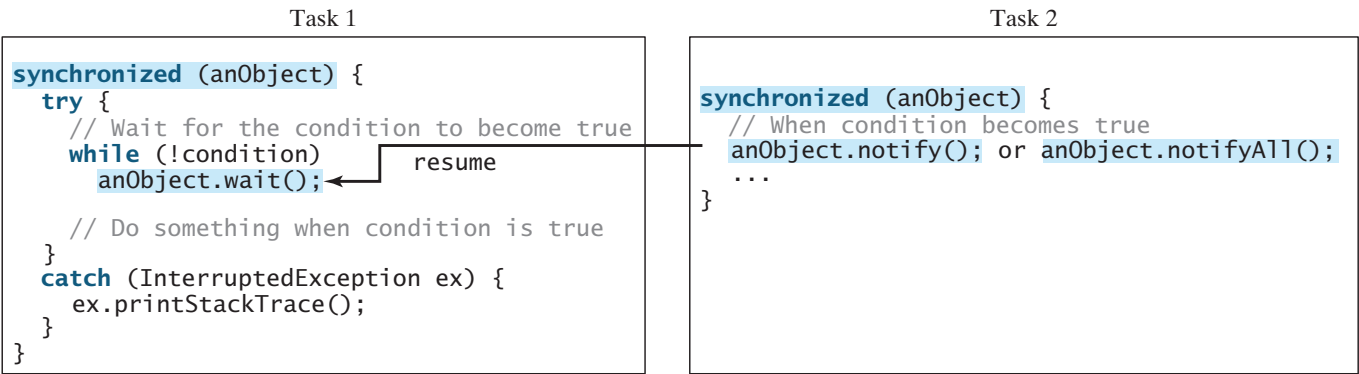


FIGURE 32.18 The `wait()`, `notify()`, and `notifyAll()` methods coordinate thread communication.

The `wait()`, `notify()`, and `notifyAll()` methods must be called in a synchronized method or a synchronized block on the receiving object of these methods. Otherwise, an `IllegalMonitorStateException` will occur.

When `wait()` is invoked, it pauses the thread and simultaneously releases the lock on the object. When the thread is restarted after being notified, the lock is automatically reacquired.

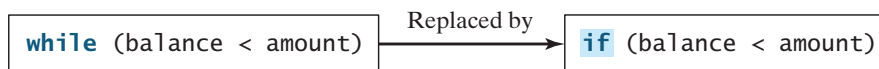
The `wait()`, `notify()`, and `notifyAll()` methods on an object are analogous to the `await()`, `signal()`, and `signalAll()` methods on a condition.

**32.19** How do you create a condition on a lock? What are the `await()`, `signal()`, and `signalAll()` methods for?



**32.20** What would happen if the `while` loop in line 58 of Listing 32.9 were changed to an `if` statement?

MyProgrammingLab™



**32.21** Why does the following class have a syntax error?

```

1  import javax.swing.*;
2
3  public class Test extends JApplet implements Runnable {
4      public void init() throws InterruptedException {
5          Thread thread = new Thread(this);
6          thread.sleep(1000);
7      }
8
9      public synchronized void run() {
10     }
11 }
  
```

**32.22** What is a possible cause for `IllegalMonitorStateException`?

**32.23** Can the `wait()`, `notify()`, and `notifyAll()` be invoked from any object? What is the purpose of these methods?

**32.24** What is wrong in the following code?

```

synchronized (object1) {
    try {
        while (!condition) object2.wait();
    }
    catch (InterruptedException ex) {
    }
}
  
```

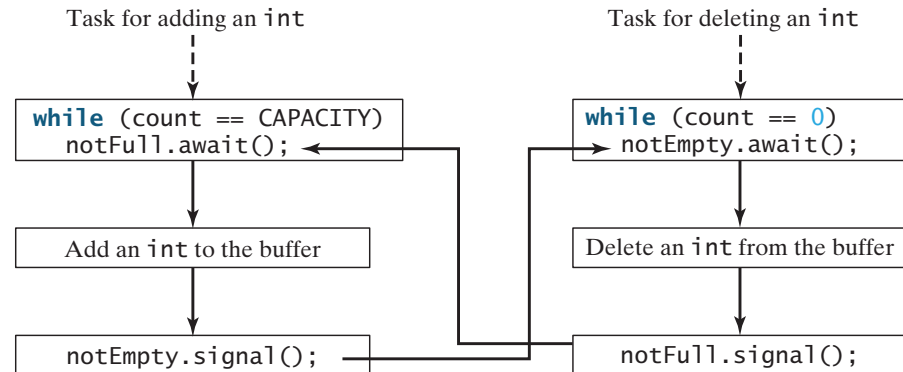
## 32.12 Case Study: Producer/Consumer

*This section gives the classic Consumer/Producer example for demonstrating thread coordination.*



Suppose you use a buffer to store integers, and that the buffer size is limited. The buffer provides the method `write(int)` to add an `int` value to the buffer and the method `read()` to read and delete an `int` value from the buffer. To synchronize the operations, use a lock with two conditions: `notEmpty` (i.e., the buffer is not empty) and `notFull` (i.e., the buffer is not full). When a task adds an `int` to the buffer, if the buffer is full, the task will wait for the `notFull` condition. When a task deletes an `int` from the buffer, if the buffer is empty, the

task will wait for the **notEmpty** condition. The interaction between the two tasks is shown in Figure 32.19.



**FIGURE 32.19** The conditions **notFull** and **notEmpty** are used to coordinate task interactions.

Listing 32.10 presents the complete program. The program contains the **Buffer** class (lines 48–95) and two tasks for repeatedly adding and consuming numbers to and from the buffer (lines 16–45). The **write(int)** method (line 60) adds an integer to the buffer. The **read()** method (line 77) deletes and returns an integer from the buffer.

The buffer is actually a first-in, first-out queue (lines 50–51). The conditions **notEmpty** and **notFull** on the lock are created in lines 57–58. The conditions are bound to a lock. A lock must be acquired before a condition can be applied. If you use the **wait()** and **notify()** methods to rewrite this example, you have to designate two objects as monitors.

### LISTING 32.10 ConsumerProducer.java

```

1  import java.util.concurrent.*;
2  import java.util.concurrent.locks.*;
3
4  public class ConsumerProducer {
5      private static Buffer buffer = new Buffer();
6
7      public static void main(String[] args) {
8          // Create a thread pool with two threads
9          ExecutorService executor = Executors.newFixedThreadPool(2);
10         executor.execute(new ProducerTask());
11         executor.execute(new ConsumerTask());
12         executor.shutdown();
13     }
14
15     // A task for adding an int to the buffer
16     private static class ProducerTask implements Runnable {
17         public void run() {
18             try {
19                 int i = 1;
20                 while (true) {
21                     System.out.println("Producer writes " + i);
22                     buffer.write(i++); // Add a value to the buffer
23                     // Put the thread to sleep
24                     Thread.sleep((int)(Math.random() * 10000));
25                 }
26             } catch (InterruptedException ex) {

```

create a buffer

create two threads

producer task

```

27         ex.printStackTrace();
28     }
29 }
30 }
31
32 // A task for reading and deleting an int from the buffer
33 private static class ConsumerTask implements Runnable {           consumer task
34     public void run() {
35         try {
36             while (true) {
37                 System.out.println("\t\t\tConsumer reads " + buffer.read());
38                 // Put the thread to sleep
39                 Thread.sleep((int)(Math.random() * 10000));
40             }
41         } catch (InterruptedException ex) {
42             ex.printStackTrace();
43         }
44     }
45 }
46
47 // An inner class for buffer
48 private static class Buffer {
49     private static final int CAPACITY = 1; // buffer size
50     private java.util.LinkedList<Integer> queue =
51         new java.util.LinkedList<Integer>();
52
53     // Create a new lock
54     private static Lock lock = new ReentrantLock();                create a lock
55
56     // Create two conditions
57     private static Condition notEmpty = lock.newCondition();       create a condition
58     private static Condition notFull = lock.newCondition();        create a condition
59
60     public void write(int value) {
61         lock.lock(); // Acquire the lock                            acquire the lock
62         try {
63             while (queue.size() == CAPACITY) {
64                 System.out.println("Wait for notFull condition");
65                 notFull.await();                                     wait for notFull
66             }
67
68             queue.offer(value);
69             notEmpty.signal(); // Signal notEmpty condition        signal notEmpty
70         } catch (InterruptedException ex) {
71             ex.printStackTrace();
72         } finally {
73             lock.unlock(); // Release the lock                      release the lock
74         }
75     }
76
77     public int read() {
78         int value = 0;
79         lock.lock(); // Acquire the lock                            acquire the lock
80         try {
81             while (queue.isEmpty()) {
82                 System.out.println("\t\t\tWait for notEmpty condition");
83                 notEmpty.await();                                   wait for notEmpty
84             }
85
86             value = queue.remove();

```

```

signal notFull      87         notFull.signal(); // Signal notFull condition
                   88     } catch (InterruptedException ex) {
                   89         ex.printStackTrace();
release the lock    90     } finally {
                   91         lock.unlock(); // Release the lock
                   92         return value;
                   93     }
                   94 }
                   95 }
                   96 }

```

A sample run of the program is shown in Figure 32.20.

**FIGURE 32.20** Locks and conditions are used for communications between the Producer and Consumer threads.



MyProgrammingLab™

**32.25** Can the **read** and **write** methods in the **Buffer** class be executed concurrently?

**32.26** When invoking the **read** method, what happens if the queue is empty?

**32.27** When invoking the **write** method, what happens if the queue is full?

## 32.13 Blocking Queues



*Java Collections Framework provides **ArrayBlockingQueue**, **LinkedBlockingQueue**, and **PriorityBlockingQueue** for supporting blocking queues.*

blocking queue

Queues and priority queues were introduced in Section 22.9. A *blocking queue* causes a thread to block when you try to add an element to a full queue or to remove an element from an empty queue. The **BlockingQueue** interface extends **java.util.Queue** and provides the synchronized **put** and **take** methods for adding an element to the tail of the queue and for removing an element from the head of the queue, as shown in Figure 32.21.

Three concrete blocking queues—**ArrayBlockingQueue**, **LinkedBlockingQueue**, and **PriorityBlockingQueue**—are provided in Java, as shown in Figure 32.22. All are in the **java.util.concurrent** package. **ArrayBlockingQueue** implements a blocking queue using an array. You have to specify a capacity or an optional fairness to construct an **ArrayBlockingQueue**. **LinkedBlockingQueue** implements a blocking queue using a linked list. You can create an unbounded or bounded **LinkedBlockingQueue**. **PriorityBlockingQueue** is a priority queue. You can create an unbounded or bounded priority queue.

unbounded queue



### Note

The **put** method will never block an unbounded **LinkedBlockingQueue** or **PriorityBlockingQueue**.

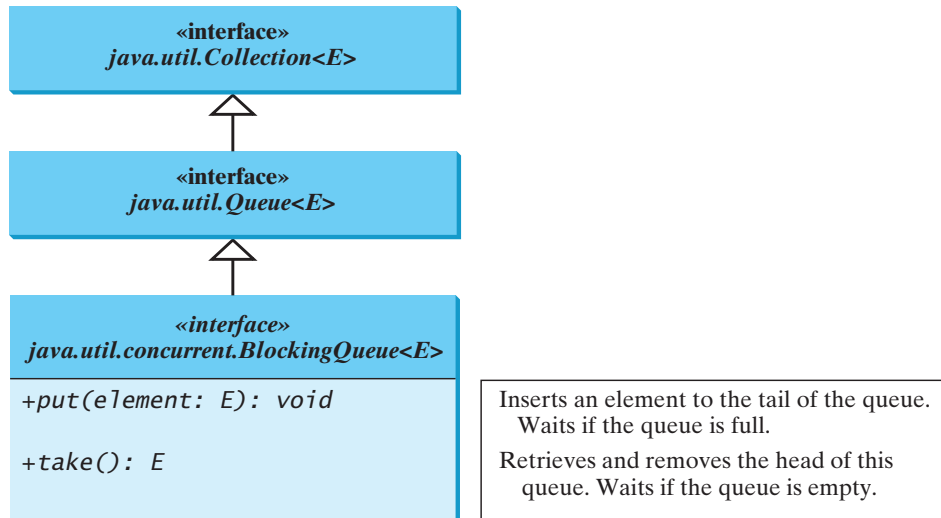


FIGURE 32.21 **BlockingQueue** is a subinterface of **Queue**.

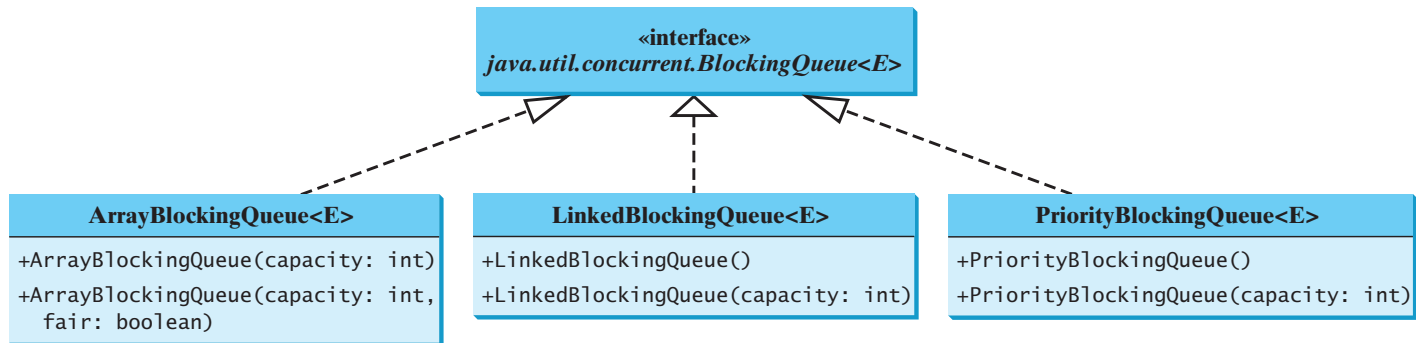


FIGURE 32.22 **ArrayBlockingQueue**, **LinkedBlockingQueue**, and **PriorityBlockingQueue** are concrete blocking queues.

Listing 32.11 gives an example of using an **ArrayBlockingQueue** to simplify the Consumer/Producer example in Listing 32.10. Line 5 creates an **ArrayBlockingQueue** to store integers. The Producer thread puts an integer into the queue (line 22), and the Consumer thread takes an integer from the queue (line 37).

### LISTING 32.11 ConsumerProducerUsingBlockingQueue.java

```

1  import java.util.concurrent.*;
2
3  public class ConsumerProducerUsingBlockingQueue {
4      private static ArrayBlockingQueue<Integer> buffer =
5          new ArrayBlockingQueue<Integer>(2);                                create a buffer
6
7      public static void main(String[] args) {
8          // Create a thread pool with two threads
9          ExecutorService executor = Executors.newFixedThreadPool(2);        create two threads
10         executor.execute(new ProducerTask());
11         executor.execute(new ConsumerTask());
12         executor.shutdown();
13     }
14
15     // A task for adding an int to the buffer
  
```

```

producer task    16  private static class ProducerTask implements Runnable {
                  17      public void run() {
                  18          try {
                  19              int i = 1;
                  20              while (true) {
put              21                  System.out.println("Producer writes " + i);
                  22                  buffer.put(i++); // Add any value to the buffer, say, 1
                  23                  // Put the thread to sleep
                  24                  Thread.sleep((int)(Math.random() * 10000));
                  25              }
                  26          } catch (InterruptedException ex) {
                  27              ex.printStackTrace();
                  28          }
                  29      }
                  30  }
                  31
consumer task    32  // A task for reading and deleting an int from the buffer
                  33  private static class ConsumerTask implements Runnable {
                  34      public void run() {
take            35          try {
                  36              while (true) {
                  37                  System.out.println("\t\t\tConsumer reads " + buffer.take());
                  38                  // Put the thread to sleep
                  39                  Thread.sleep((int)(Math.random() * 10000));
                  40              }
                  41          } catch (InterruptedException ex) {
                  42              ex.printStackTrace();
                  43          }
                  44      }
                  45  }
                  46  }

```

In Listing 32.10, you used locks and conditions to synchronize the Producer and Consumer threads. In this program, hand coding is not necessary, because synchronization is already implemented in [ArrayBlockingQueue](#).



MyProgrammingLab™

**32.28** What is a blocking queue? What blocking queues are supported in Java?

**32.29** What method do you use to add an element to an [ArrayBlockingQueue](#)? What happens if the queue is full?

**32.30** What method do you use to retrieve an element from an [ArrayBlockingQueue](#)? What happens if the queue is empty?

## 32.14 Semaphores

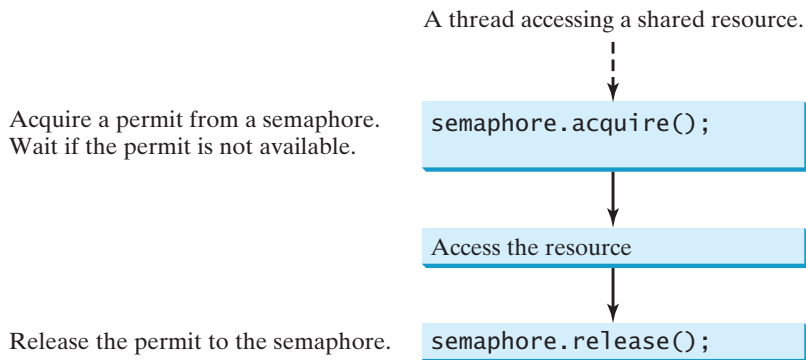


*Semaphores can be used to restrict the number of threads that access a shared resource.*

semaphore

In computer science, a *semaphore* is an object that controls the access to a common resource. Before accessing the resource, a thread must acquire a permit from the semaphore. After finishing with the resource, the thread must return the permit back to the semaphore, as shown in Figure 32.23.

To create a semaphore, you have to specify the number of permits with an optional fairness policy, as shown in Figure 32.24. A task acquires a permit by invoking the semaphore's [acquire\(\)](#) method and releases the permit by invoking the semaphore's [release\(\)](#) method. Once a permit is acquired, the total number of available permits in a semaphore is reduced by 1. Once a permit is released, the total number of available permits in a semaphore is increased by 1.



**FIGURE 32.23** A limited number of threads can access a shared resource controlled by a semaphore.

| <code>java.util.concurrent.Semaphore</code>                  |                                                                                                                 |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>+Semaphore(numberOfPermits: int)</code>                | Creates a semaphore with the specified number of permits. The fairness policy is false.                         |
| <code>+Semaphore(numberOfPermits: int, fair: boolean)</code> | Creates a semaphore with the specified number of permits and the fairness policy.                               |
| <code>+acquire(): void</code>                                | Acquires a permit from this semaphore. If no permit is available, the thread is blocked until one is available. |
| <code>+release(): void</code>                                | Releases a permit back to the semaphore.                                                                        |

**FIGURE 32.24** The `Semaphore` class contains the methods for accessing a semaphore.

A semaphore with just one permit can be used to simulate a mutually exclusive lock. Listing 32.12 revises the `Account` inner class in Listing 32.9 using a semaphore to ensure that only one thread at a time can access the `deposit` method.

### LISTING 32.12 New Account Inner Class

```

1 // An inner class for Account
2 private static class Account {
3     // Create a semaphore
4     private static Semaphore semaphore = new Semaphore(1);
5     private int balance = 0;
6
7     public int getBalance() {
8         return balance;
9     }
10
11     public void deposit(int amount) {
12         try {
13             semaphore.acquire(); // Acquire a permit
14             int newBalance = balance + amount;
15
16             // This delay is deliberately added to magnify the
17             // data-corruption problem and make it easy to see
18             Thread.sleep(5);
19
20             balance = newBalance;
21         }
22         catch (InterruptedException ex) {

```

create a semaphore

acquire a permit

release a permit

```

23     }
24     finally {
25         semaphore.release(); // Release a permit
26     }
27 }
28 }

```

A semaphore with one permit is created in line 4. A thread first acquires a permit when executing the deposit method in line 13. After the balance is updated, the thread releases the permit in line 25. It is a good practice to always place the `release()` method in the `finally` clause to ensure that the permit is finally released even in the case of exceptions.



MyProgrammingLab™

**32.31** What are the similarities and differences between a lock and a semaphore?

**32.32** How do you create a semaphore that allows three concurrent threads? How do you acquire a semaphore? How do you release a semaphore?

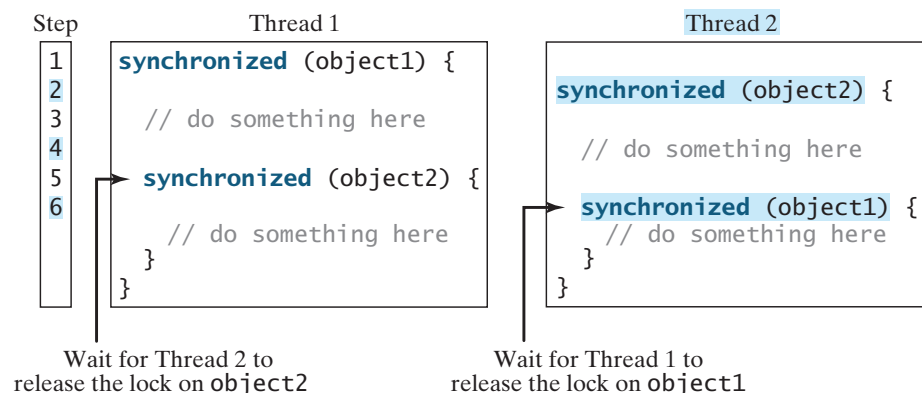
## 32.15 Avoiding Deadlocks



deadlock

*Deadlocks can be avoided by using a proper resource ordering.*

Sometimes two or more threads need to acquire the locks on several shared objects. This could cause a *deadlock*, in which each thread has the lock on one of the objects and is waiting for the lock on the other object. Consider the scenario with two threads and two objects, as shown in Figure 32.25. Thread 1 has acquired a lock on `object1`, and Thread 2 has acquired a lock on `object2`. Now Thread 1 is waiting for the lock on `object2`, and Thread 2 for the lock on `object1`. Each thread waits for the other to release the lock it needs, and until that happens, neither can continue to run.



**FIGURE 32.25** Thread 1 and Thread 2 are deadlocked.

resource ordering

Deadlock is easily avoided by using a simple technique known as *resource ordering*. With this technique, you assign an order to all the objects whose locks must be acquired and ensure that each thread acquires the locks in that order. For the example in Figure 32.25, suppose that the objects are ordered as `object1` and `object2`. Using the resource ordering technique, Thread 2 must acquire a lock on `object1` first, then on `object2`. Once Thread 1 acquires a lock on `object1`, Thread 2 has to wait for a lock on `object1`. Thus, Thread 1 will be able to acquire a lock on `object2` and no deadlock will occur.



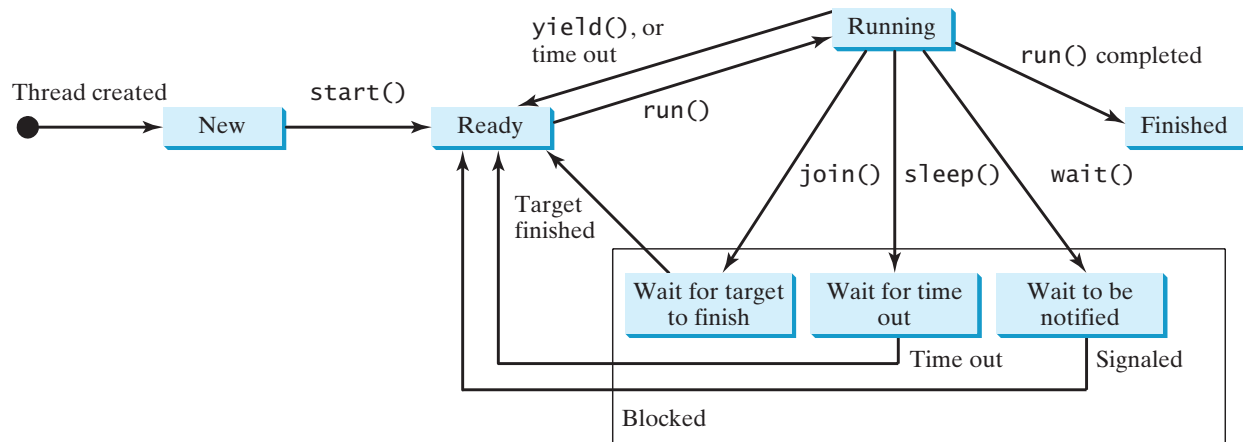
MyProgrammingLab™

**32.33** What is a deadlock? How can you avoid deadlock?

## 32.16 Thread States

A thread state indicates the status of thread.

Tasks are executed in threads. Threads can be in one of five states: New, Ready, Running, Blocked, or Finished (see Figure 32.26).



**FIGURE 32.26** A thread can be in one of five states: New, Ready, Running, Blocked, or Finished.

When a thread is newly created, it enters the **New** state. After a thread is started by calling its **start()** method, it enters the **Ready** state. A ready thread is runnable but may not be running yet. The operating system has to allocate CPU time to it.

When a ready thread begins executing, it enters the **Running** state. A running thread can enter the **Ready** state if its given CPU time expires or its **yield()** method is called.

A thread can enter the **Blocked** state (i.e., become inactive) for several reasons. It may have invoked the **join()**, **sleep()**, or **wait()** method. It may be waiting for an I/O operation to finish. A blocked thread may be reactivated when the action inactivating it is reversed. For example, if a thread has been put to sleep and the sleep time has expired, the thread is reactivated and enters the **Ready** state.

Finally, a thread is **Finished** if it completes the execution of its **run()** method.

The **isAlive()** method is used to find out the state of a thread. It returns **true** if a thread is in the **Ready**, **Blocked**, or **Running** state; it returns **false** if a thread is new and has not started or if it is finished.

The **interrupt()** method interrupts a thread in the following way: If a thread is currently in the **Ready** or **Running** state, its interrupted flag is set; if a thread is currently blocked, it is awakened and enters the **Ready** state, and a **java.lang.InterruptedException** is thrown.

**32.34** What is a thread state? Describe the states for a thread.



## 32.17 Synchronized Collections

Java Collections Framework provides synchronized collections for lists, sets, and maps.

MyProgrammingLab™



The classes in the Java Collections Framework are not thread-safe; that is, their contents may become corrupted if they are accessed and updated concurrently by multiple threads. You can protect the data in a collection by locking the collection or by using synchronized collections.

synchronized collection

The **Collections** class provides six static methods for wrapping a collection into a synchronized version, as shown in Figure 32.27. The collections created using these methods are called *synchronization wrappers*.

| java.util.Collections                              |                                                                  |
|----------------------------------------------------|------------------------------------------------------------------|
| +synchronizedCollection(c: Collection): Collection | Returns a synchronized collection.                               |
| +synchronizedList(list: List): List                | Returns a synchronized list from the specified list.             |
| +synchronizedMap(m: Map): Map                      | Returns a synchronized map from the specified map.               |
| +synchronizedSet(s: Set): Set                      | Returns a synchronized set from the specified set.               |
| +synchronizedSortedMap(s: SortedMap): SortedMap    | Returns a synchronized sorted map from the specified sorted map. |
| +synchronizedSortedSet(s: SortedSet): SortedSet    | Returns a synchronized sorted set.                               |

**FIGURE 32.27** You can obtain synchronized collections using the methods in the **Collections** class.

Invoking **synchronizedCollection(Collection c)** returns a new **Collection** object, in which all the methods that access and update the original collection **c** are synchronized. These methods are implemented using the **synchronized** keyword. For example, the **add** method is implemented like this:

```
public boolean add(E o) {
    synchronized (this) {
        return c.add(o);
    }
}
```

Synchronized collections can be safely accessed and modified by multiple threads concurrently.



#### Note

The methods in **java.util.Vector**, **java.util.Stack**, and **java.util.Hashtable** are already synchronized. These are old classes introduced in JDK 1.0. Starting with JDK 1.5, you should use **java.util.ArrayList** to replace **Vector**, **java.util.LinkedList** to replace **Stack**, and **java.util.Map** to replace **Hashtable**. If synchronization is needed, use a synchronization wrapper.

fail-fast

The synchronization wrapper classes are thread-safe, but the iterator is *fail-fast*. This means that if you are using an iterator to traverse a collection while the underlying collection is being modified by another thread, then the iterator will immediately fail by throwing **java.util.ConcurrentModificationException**, which is a subclass of **RuntimeException**. To avoid this error, you need to create a synchronized collection object and acquire a lock on the object when traversing it. For example, to traverse a set, you have to write the code like this:

```
Set hashSet = Collections.synchronizedSet(new HashSet());

synchronized (hashSet) { // Must synchronize it
    Iterator iterator = hashSet.iterator();

    while (iterator.hasNext()) {
        System.out.println(iterator.next());
    }
}
```

Failure to do so may result in nondeterministic behavior, such as a **ConcurrentModificationException**.

**32.35** What is a synchronized collection? Is **ArrayList** synchronized? How do you make it synchronized?

**32.36** Explain why an iterator is fail-fast.



MyProgrammingLab™

## 32.18 Parallel Programming

*The Fork/Join Framework is used for parallel programming in Java.*

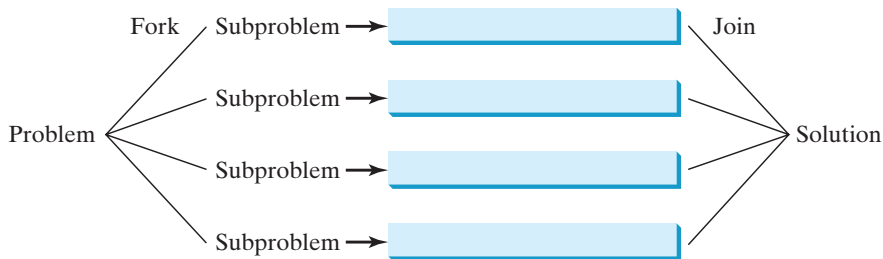
The widespread use of multicore systems has created a revolution in software. In order to benefit from multiple processors, software needs to run in parallel. JDK 7 introduces the new Fork/Join Framework for parallel programming, which utilizes the multicore processors.

The *Fork/Join Framework* is illustrated in Figure 32.28 (the diagram resembles a fork, hence its name). A problem is divided into nonoverlapping subproblems, which can be solved independently in parallel. The solutions to all subproblems are then joined to obtain the overall solution for the problem. This is the parallel implementation of the divide-and-conquer approach. In JDK 7's Fork/Join Framework, a *fork* can be viewed as an independent task that runs on a thread.



JDK 7 feature

Fork/Join Framework



**FIGURE 32.28** The nonoverlapping subproblems are solved in parallel.

The framework defines a task using the **ForkJoinTask** class, as shown in Figure 32.29, and executes a task in an instance of **ForkJoinPool**, as shown in Figure 32.30.

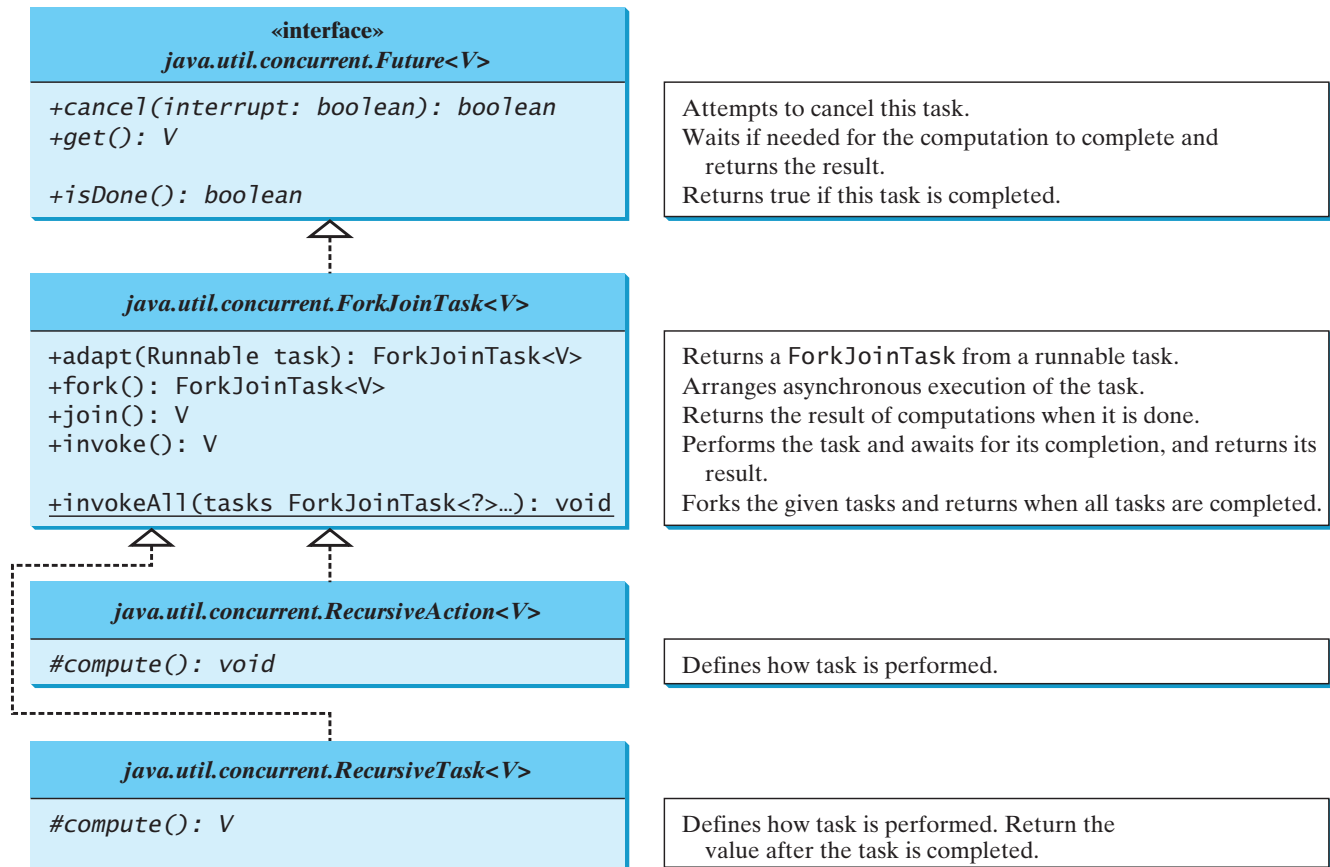
**ForkJoinTask** is the abstract base class for tasks. A **ForkJoinTask** is a thread-like entity, but it is much lighter than a normal thread, because huge numbers of tasks and subtasks can be executed by a small number of actual threads in a **ForkJoinPool**. The tasks are primarily coordinated using **fork()** and **join()**. Invoking **fork()** on a task arranges asynchronous execution, and invoking **join()** waits until the task is completed. The **invoke()** and **invokeAll(tasks)** methods implicitly invoke **fork()** to execute the task and **join()** to wait for the tasks to complete, and return the result, if any. Note that the static method **invokeAll** takes a variable number of **ForkJoinTask** arguments using the **...** syntax, which was introduced in Section 6.9.

The Fork/Join Framework is designed to parallelize divide-and-conquer solutions, which are naturally recursive. **RecursiveAction** and **RecursiveTask** are two subclasses of **ForkJoinTask**. To define a concrete task class, your class should extend **RecursiveAction** or **RecursiveTask**. **RecursiveAction** is for a task that doesn't return a value, and **RecursiveTask** is for a task that does return a value. Your task class should override the **compute()** method to specify how a task is performed.

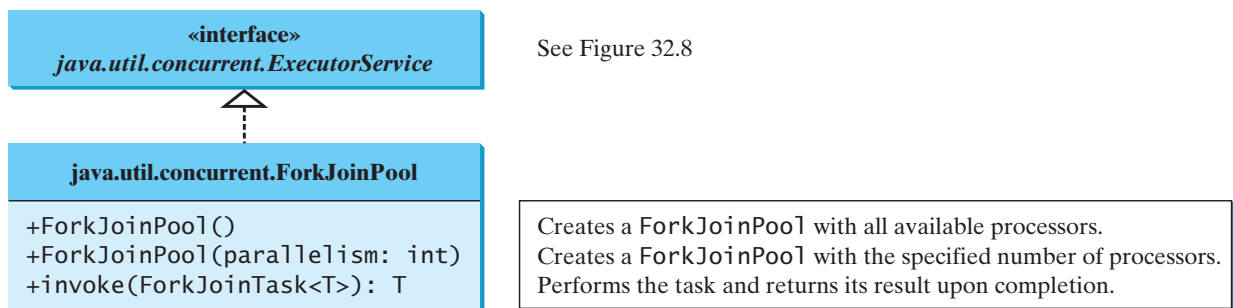
We now use a merge sort to demonstrate how to develop parallel programs using the Fork/Join Framework. The merge sort algorithm (introduced in Section 25.3) divides the

ForkJoinTask  
ForkJoinPool

RecursiveAction  
RecursiveTask



**FIGURE 32.29** The `ForkJoinTask` class defines a task for asynchronous execution.



**FIGURE 32.30** The `ForkJoinPool` executes Fork/Join tasks.

array into two halves and applies a merge sort on each half recursively. After the two halves are sorted, the algorithm merges them. Listing 32.13 gives a parallel implementation of the merge sort algorithm and compares its execution time with a sequential sort.

### LISTING 32.13 ParallelMergeSort.java

```

1 import java.util.concurrent.RecursiveAction;
2 import java.util.concurrent.ForkJoinPool;
3
4 public class ParallelMergeSort {
5     public static void main(String[] args) {

```

```

6      final int SIZE = 7000000;
7      int[] list1 = new int[SIZE];
8      int[] list2 = new int[SIZE];
9
10     for (int i = 0; i < list1.length; i++)
11         list1[i] = list2[i] = (int)(Math.random() * 10000000);
12
13     long startTime = System.currentTimeMillis();
14     parallelMergeSort(list1); // Invoke parallel merge sort           invoke parallel sort
15     long endTime = System.currentTimeMillis();
16     System.out.println("\nParallel time with "
17         + Runtime.getRuntime().availableProcessors() +
18         " processors is " + (endTime - startTime) + " milliseconds");
19
20     startTime = System.currentTimeMillis();
21     MergeSort.mergeSort(list2); // MergeSort is in Listing 25.5     invoke sequential sort
22     endTime = System.currentTimeMillis();
23     System.out.println("\nSequential time is " +
24         (endTime - startTime) + " milliseconds");
25 }
26
27 public static void parallelMergeSort(int[] list) {
28     RecursiveAction mainTask = new SortTask(list);
29     ForkJoinPool pool = new ForkJoinPool();
30     pool.invoke(mainTask);
31 }
32
33 private static class SortTask extends RecursiveAction {
34     private final int THRESHOLD = 500;
35     private int[] list;
36
37     SortTask(int[] list) {
38         this.list = list;
39     }
40
41     @Override
42     protected void compute() {
43         if (list.length < THRESHOLD)
44             java.util.Arrays.sort(list);
45         else {
46             // Obtain the first half
47             int[] firstHalf = new int[list.length / 2];
48             System.arraycopy(list, 0, firstHalf, 0, list.length / 2);
49
50             // Obtain the second half
51             int secondHalfLength = list.length - list.length / 2;
52             int[] secondHalf = new int[secondHalfLength];
53             System.arraycopy(list, list.length / 2,
54                 secondHalf, 0, secondHalfLength);
55
56             // Recursively sort the two halves
57             invokeAll(new SortTask(firstHalf),
58                 new SortTask(secondHalf));
59
60             // Merge firstHalf with secondHalf into list
61             MergeSort.merge(firstHalf, secondHalf, list);
62         }
63     }
64 }
65 }

```

create a ForkJoinTask  
create a ForkJoinPool  
execute a task

define concrete  
ForkJoinTask

perform the task

sort a small list

split into two parts

solve each part

merge two parts



Parallel time with 2 processors is 2829 milliseconds  
Sequential time is 4751 milliseconds

Since the sort algorithm does not return a value, we define a concrete `ForkJoinTask` class by extending `RecursiveAction` (lines 33–64). The `compute` method is overridden to implement a recursive merge sort (lines 42–63). If the list is small, it is more efficient to be solved sequentially (line 44). For a large list, it is split into two halves (lines 47–54). The two halves are sorted concurrently (lines 57–58) and then merged (line 61).

The program creates a main `ForkJoinTask` (line 28), a `ForkJoinPool` (line 29), and places the main task for execution in a `ForkJoinPool` (line 30). The `invoke` method will return after the main task is completed.

When executing the main task, the task is split into subtasks and the subtasks are invoked using the `invokeAll` method (lines 57–58). The `invokeAll` method will return after all the subtasks are completed. Note that each subtask is further split into smaller tasks recursively. Huge numbers of subtasks may be created and executed in the pool. The Fork/Join Framework automatically executes and coordinates all the tasks efficiently.

The `MergeSort` class is defined in Listing 25.5. The program invokes `MergeSort.merge` to merge two sorted sublists (line 61). The program also invokes `MergeSort.mergeSort` (line 21) to sort a list using merge sort sequentially. You can see that the parallel sort is much faster than the sequential sort.

Note that the loop for initializing the list can also be parallelized. However, you should avoid using `Math.random()` in the code, because it is synchronized and cannot be executed in parallel (see Programming Exercise 32.12). The `parallelMergeSort` method only sorts an array of `int` values, but you can modify it to become a generic method (see Programming Exercise 32.13).

In general, a problem can be solved in parallel using the following pattern:

```
if (the program is small)
    solve it sequentially;
else {
    divide the problem into nonoverlapping subproblems;
    solve the subproblems concurrently;
    combine the results from subproblems to solve the whole problem;
}
```

Listing 32.14 develops a parallel method that finds the maximal number in a list.

### LISTING 32.14 ParallelMax.java

```
1  import java.util.concurrent.*;
2
3  public class ParallelMax {
4      public static void main(String[] args) {
5          // Create a list
6          final int N = 9000000;
7          int[] list = new int[N];
8          for (int i = 0; i < list.length; i++)
9              list[i] = i;
10
11         long startTime = System.currentTimeMillis();
12         System.out.println("\nThe maximal number is " + max(list));
13         long endTime = System.currentTimeMillis();
14         System.out.println("The number of processors is " +
15             Runtime.getRuntime().availableProcessors());
16         System.out.println("Time is " + (endTime - startTime))
```

invoke max

```

17         + " milliseconds");
18     }
19
20     public static int max(int[] list) {
21         RecursiveTask<Integer> task = new MaxTask(list, 0, list.length);
22         ForkJoinPool pool = new ForkJoinPool();
23         return pool.invoke(task);
24     }
25
26     private static class MaxTask extends RecursiveTask<Integer> {
27         private final static int THRESHOLD = 1000;
28         private int[] list;
29         private int low;
30         private int high;
31
32         public MaxTask(int[] list, int low, int high) {
33             this.list = list;
34             this.low = low;
35             this.high = high;
36         }
37
38         @Override
39         public Integer compute() {
40             if (high - low < THRESHOLD) {
41                 int max = list[0];
42                 for (int i = low; i < high; i++)
43                     if (list[i] > max)
44                         max = list[i];
45                 return new Integer(max);
46             }
47             else {
48                 int mid = (low + high) / 2;
49                 RecursiveTask<Integer> left = new MaxTask(list, low, mid);
50                 RecursiveTask<Integer> right = new MaxTask(list, mid, high);
51
52                 right.fork();
53                 left.fork();
54                 return new Integer(Math.max(left.join().intValue(),
55                     right.join().intValue()));
56             }
57         }
58     }
59 }

```

create a ForkJoinTask  
create a ForkJoinPool  
execute a task

define concrete ForkJoinTask

perform the task

solve a small problem

split into two parts

fork right  
fork left  
join tasks

The maximal number is 8999999  
The number of processors is 2  
Time is 44 milliseconds



Since the algorithm returns an integer, we define a task class for fork join by extending `RecursiveTask<Integer>` (lines 26–58). The `compute` method is overridden to return the max element in a `list[low..high]` (lines 39–57). If the list is small, it is more efficient to be solved sequentially (lines 40–46). For a large list, it is split into two halves (lines 48–50). The tasks `left` and `right` find the maximal element in the left half and right half, respectively. Invoking `fork()` on the task causes the task to be executed (lines 52–53). The `join()` method awaits for the task to complete and then returns the result (lines 54–55).



MyProgrammingLab™

- 32.37** How do you define a **ForkJoinTask**? What are the differences between **RecursiveAction** and **RecursiveTask**?
- 32.38** How do you tell the system to execute a task?
- 32.39** What method can you use to test if a task has been completed?
- 32.40** How do you create a **ForkJoinPool**? How do you place a task into a **ForkJoinPool**?

## KEY TERMS

|                            |                              |
|----------------------------|------------------------------|
| condition 1150             | multithreading 1130          |
| deadlock 1162              | race condition 1147          |
| event dispatch thread 1138 | semaphore 1160               |
| fail-fast 1164             | synchronization wrapper 1164 |
| fairness policy 1149       | synchronized block 1148      |
| Fork/Join Framework 1165   | thread 1130                  |
| lock 1148                  | thread-safe 1147             |
| monitor 1154               |                              |

## CHAPTER SUMMARY

- Each task is an instance of the **Runnable** interface. A *thread* is an object that facilitates the execution of a task. You can define a task class by implementing the **Runnable** interface and create a thread by wrapping a task using a **Thread** constructor.
- After a thread object is created, use the **start()** method to start a thread, and the **sleep(long)** method to put a thread to sleep so that other threads get a chance to run.
- A thread object never directly invokes the **run** method. The JVM invokes the **run** method when it is time to execute the thread. Your class must override the **run** method to tell the system what the thread will do when it runs.
- To prevent threads from corrupting a shared resource, use *synchronized* methods or blocks. A *synchronized method* acquires a *lock* before it executes. In the case of an instance method, the lock is on the object for which the method was invoked. In the case of a static (class) method, the lock is on the class.
- A synchronized statement can be used to acquire a lock on any object, not just *this* object, when executing a block of the code in a method. This block is referred to as a *synchronized block*.
- You can use explicit locks and *conditions* to facilitate communications among threads, as well as using the built-in monitor for objects.
- Deadlock* occurs when two or more threads acquire locks on multiple objects and each has a lock on one object and is waiting for the lock on the other object. The *resource ordering technique* can be used to avoid deadlock.
- The JDK 7's Fork/Join Framework is designed for developing parallel programs. You can define a task class that extends **RecursiveAction** or **RecursiveTask** and execute the tasks concurrently in **ForkJoinPool**, and obtains the overall solution after all tasks are completed.

## TEST QUESTIONS

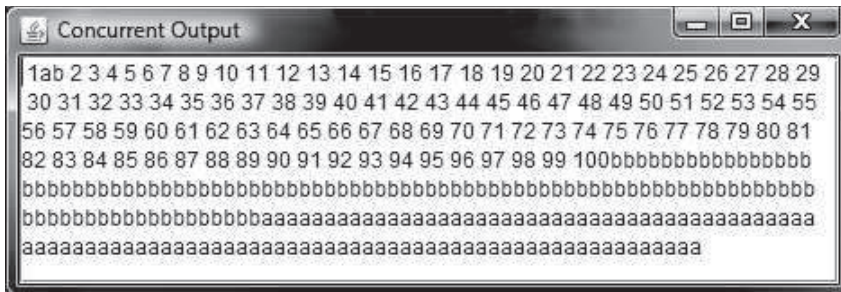
Do the test questions for this chapter online at [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

## PROGRAMMING EXERCISES

MyProgrammingLab™

### Sections 32.1–32.5

- \*32.1** (*Revise Listing 32.1*) Rewrite Listing 32.1 to display the output in a text area, as shown in Figure 32.31.



**FIGURE 32.31** The output from three threads is displayed in a text area.

- 32.2** (*Racing cars*) Rewrite Programming Exercise 18.17 using a thread to control car racing. Compare the program with Programming Exercise 18.17 by setting the delay time to 10 in both programs. Which one runs the animation faster?
- 32.3** (*Raise flags*) Rewrite Programming Exercise 18.23 using a thread to animate a flag being raised. Compare the program with Programming Exercise 18.23 by setting the delay time to 10 in both programs. Which one runs the animation faster?

### Sections 32.8–32.12

- 32.4** (*Synchronize threads*) Write a program that launches 1,000 threads. Each thread adds **1** to a variable **sum** that initially is **0**. You need to pass **sum** by reference to each thread. In order to pass it by reference, define an **Integer** wrapper object to hold **sum**. Run the program with and without synchronization to see its effect.
- 32.5** (*Run fans*) Rewrite Programming Exercise 18.11 using a thread to control the fan animation.
- 32.6** (*Bouncing balls*) Rewrite Programming Exercise 18.19 using a thread to animate bouncing ball movements.
- 32.7** (*Control a group of clocks*) Rewrite Programming Exercise 18.14 using a thread to control the clock animation.
- 32.8** (*Account synchronization*) Rewrite Listing 32.9, `ThreadCooperation.java`, using the object's `wait()` and `notifyAll()` methods.
- 32.9** (*Demonstrate **ConcurrentModificationException***) The iterator is *fail-fast*. Write a program to demonstrate it by creating two threads that concurrently access and modify a set. The first thread creates a hash set filled with numbers, and adds a new number to the set every second. The second thread obtains an iterator for the set and traverses the set back and forth through the iterator every second. You will receive a **ConcurrentModificationException** because the underlying set is being modified in the first thread while the set in the second thread is being traversed.

- \*32.10** (*Use synchronized sets*) Using synchronization, correct the problem in the preceding exercise so that the second thread does not throw a `ConcurrentModificationException`.

### Section 32.15

- \*32.11** (*Demonstrate deadlock*) Write a program that demonstrates deadlock.

### Section 32.18

- \*32.12** (*Parallel array initializer*) Implement the following method using the Fork/Join Framework to assign random values to the list.

```
public static void parallelAssignValues(double[] list)
```

Write a test program that creates a list with 9,000,000 elements and invokes `parallelAssignValues` to assign random values to the list. Also implement a sequential algorithm and compare the execution time of the two. Note that if you use `Math.random()`, your parallel code execution time will be worse than the sequential code execution time, because `Math.random()` is synchronized and cannot be executed in parallel. To fix this problem, create a `Random` object for assigning random values to a small list.

- 32.13** (*Generic parallel merge sort*) Revise Listing 32.13, `ParallelMergeSort.java`, to define a generic `parallelMergeSort` method as follows:

```
public static void <E extends Comparable<E>>
    parallelMergeSort(E[] list)
```

- \*32.14** (*Parallel quick sort*) Implement the following method in parallel to sort a list using quick sort (see Listing 25.7).

```
public static void parallelQuickSort(int[] list)
```

Write a test program that times the execution time for a list of size 9,000,000 using this parallel method and a sequential method.

- \*32.15** (*Parallel sum*) Implement the following method using Fork/Join to find the sum of a list.

```
public static double parallelSum(double[] list)
```

Write a test program that finds the sum in a list of 9,000,000 double values.

- \*32.16** (*Parallel matrix addition*) Programming Exercise 7.5 describes how to perform matrix addition. Suppose you have multiple processors, so you can speed up the matrix addition. Implement the following method in parallel.

```
public static double[][] parallelAddMatrix(
    double[][] a, double[][] b)
```

Write a test program that times the execution time for adding two  $2,000 \times 2,000$  matrices.

- \*32.17** (*Parallel matrix multiplication*) Programming Exercise 7.6 describes how to perform matrix multiplication. Suppose you have multiple processors, so you can speed up the matrix multiplication. Implement the following method in parallel.

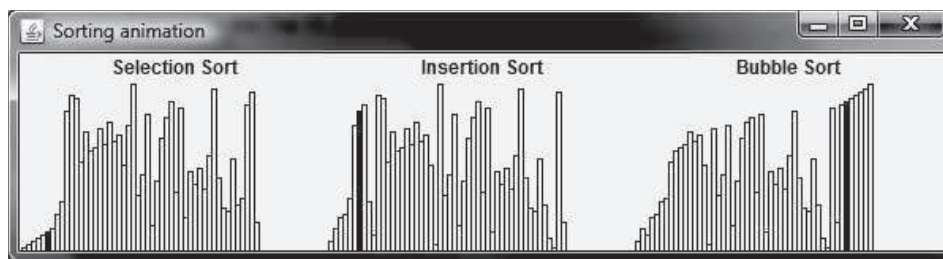
```
public static double[][] parallelMultiplyMatrix(
    double[][] a, double[][] b)
```

Write a test program that times the execution time for multiplying two  $2,000 \times 2,000$  matrices.

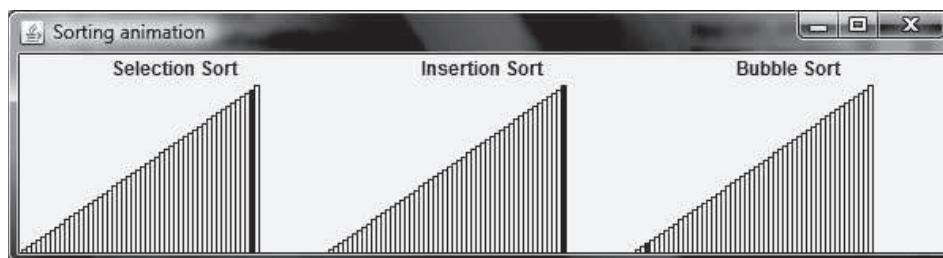
- \*32.18** (*Parallel Eight Queens*) Revise Listing 24.10, `EightQueens.java`, to develop a parallel algorithm that finds all solutions for the Eight Queens problem. (*Hint*: Launch eight subtasks, each of which places the queen in a different column in the first row.)

### Comprehensive

- \*\*\*32.19** (*Sorting animation*) Write an animation applet for selection sort, insertion sort, and bubble sort, as shown in Figure 32.32. Create an array of integers 1, 2, . . . , 50. Shuffle it randomly. Create a panel to display the array in a histogram. You should invoke each sort method in a separate thread. Each algorithm uses two nested loops. When the algorithm completes an iteration in the outer loop, put the thread to sleep for 0.5 seconds, and redisplay the array in the histogram. Color the last bar in the sorted subarray.



(a) Sorting in progress

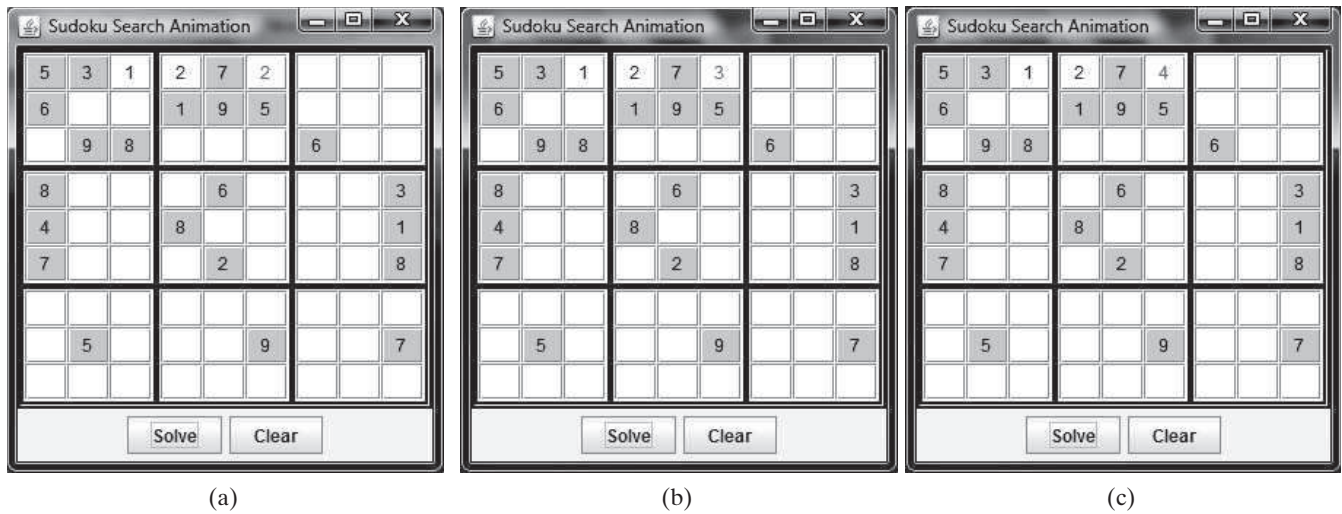


(b) Sorted

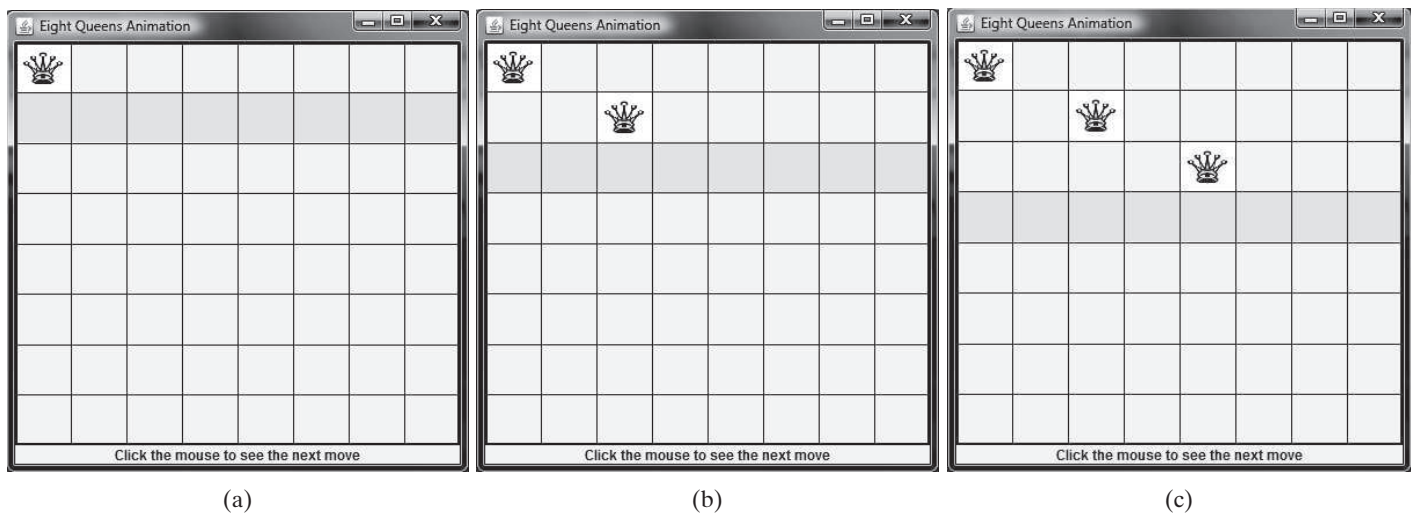
**FIGURE 32.32** Three sorting algorithms are illustrated in the animation.

- \*\*\*32.20** (*Sudoku search animation*) Modify Programming Exercise 24.21 to display the intermediate results of the search. As shown in Figure 32.33a, the number 2 is placed in the first row and last column, because 2 already appears in the same row. This number is invalid, so the next value, 3, is placed in Figure 32.33b. This number is also invalid, because 3 already appears in the same row; so the next value, 4, is placed in Figure 32.33c. The animation displays all the search steps.

- \*\*\*32.21** (*Eight Queens animation*) Modify Listing 24.10, `EightQueens.java`, to display the intermediate results of the search. As shown in Figure 32.34a, the current row being searched is highlighted. When the user clicks the mouse button, a position for the row is found and a queen is placed in the row, as shown in Figure 32.34b.



**FIGURE 32.33** The intermediate search steps are displayed in the animation for the Sudoku problem.



**FIGURE 32.34** The intermediate search steps are displayed in the animation for the Eight Queens problem.